

Universidade Federal do Rio Grande do Sul

Instituto de Informática — INF01017 Aprendizado de Máquina

Prof^a. Mariana Recamonde Mendoza**Trabalho Prático Final — Partes 1 e 2 (T1/T2)****Modelagem Preditiva com Aprendizado de Máquina: Análise, Otimização e Interpretação
do conjunto *Car Evaluation***Bernardo Oliveira, Caetano Meneghetti, Nickolas Machado

1 Coleta de dados e identificação do problema**1.1 Origem e contexto do conjunto**

O conjunto *Car Evaluation Database* foi criado por Bohanec e Rajkovič em 1988, a partir de um modelo hierárquico de decisão (DEX) construído originalmente para demonstrar sistemas especialistas multicritério. A base está disponível publicamente no UCI Machine Learning Repository sob o identificador **19**, no endereço <https://archive.ics.uci.edu/dataset/19/car+evaluation>.

Por construção, o conjunto enumera *todas* as combinações possíveis dos seis atributos preditores ($4 \cdot 4 \cdot 4 \cdot 3 \cdot 3 \cdot 3 = 1728$). A base não foi obtida por amostragem: cada uma das 1.728 instâncias do produto cartesiano dos atributos recebeu seu rótulo pela aplicação determinística do modelo DEX.

1.2 Pergunta de pesquisa e tarefa preditiva

O objetivo é construir um modelo que consiga **prever a aceitabilidade global de um automóvel** a partir de seis características descritivas. O problema é, portanto, de **classificação multiclasse** com quatro rótulos ordenados:

$$\text{unacc (inaceitável)} \prec \text{acc (aceitável)} \prec \text{good (bom)} \prec \text{vgood (muito bom)}.$$
1.3 Atributos do conjunto

Os seis atributos preditores são todos categóricos, têm ordem natural e baixa cardinalidade (Tabela 1). A classe também tem ordem natural, o que abre espaço para abordagens *ordinais* de modelagem (avaliadas mais adiante).

Tabela 1: Atributos preditores e respectivos domínios (ordem natural crescente).

Atributo	Significado	Domínio (ordem natural →)	Cardinalidade
buying	preço de compra	low, med, high, vhigh	4
maint	custo de manutenção	low, med, high, vhigh	4
doors	número de portas	2, 3, 4, 5more	4
persons	capacidade	2, 4, more	3
lug_boot	porta-malas	small, med, big	3
safety	segurança estimada	low, med, high	3

1.4 Justificativa da escolha do conjunto

Escolhemos este conjunto pelas seguintes razões:

- domínio de fácil interpretação, com atributos auto-explicativos;
- tamanho moderado ($\approx 1,7k$ instâncias), adequado ao *spot-checking* previsto no T1;
- ausência de valores faltantes e pouca necessidade de limpeza;
- presença simultânea de classes desbalanceadas e atributos ordinais, o que permite discutir vieses indutivos diferentes na fase posterior;
- por ter sido gerado a partir de um modelo de regras hierárquicas conhecido, esperamos que algoritmos baseados em árvores recuperem boa parte da estrutura — o que serve como referência útil para comparar diferentes vieses indutivos.

2 Análise exploratória e pré-processamento dos dados

2.1 Inspeção inicial e qualidade dos dados

A carga foi feita diretamente do arquivo `car.data`, atribuindo os nomes das colunas conforme `car.names` e tipando cada atributo como *categórico ordenado* respeitando sua ordem natural. A inspeção inicial revelou:

- **Dimensões:** 1.728 linhas $\times$ 7 colunas (6 atributos preditores + classe);
- **Valores faltantes:** nenhum (0/1728 em todas as colunas);
- **Linhas duplicadas:** nenhuma, o que é coerente com o fato de o conjunto ser o produto cartesiano completo;
- **Tipos:** todas as variáveis são categóricas; nenhum atributo numérico.

A combinação de dados sem ruído, completos e sem duplicatas indica que **não há necessidade de imputação ou remoção** de instâncias.

2.2 Distribuição do atributo alvo

A Figura 1 mostra a distribuição da variável `class`. A base é **fortemente desbalanceada**: cerca de 70 % das instâncias correspondem à classe `unacc`, enquanto as classes `good` e `vgood` somam menos de 8 %. A razão entre a classe majoritária e a minoritária é de aproximadamente $1210/65 \approx 18,6\times$.

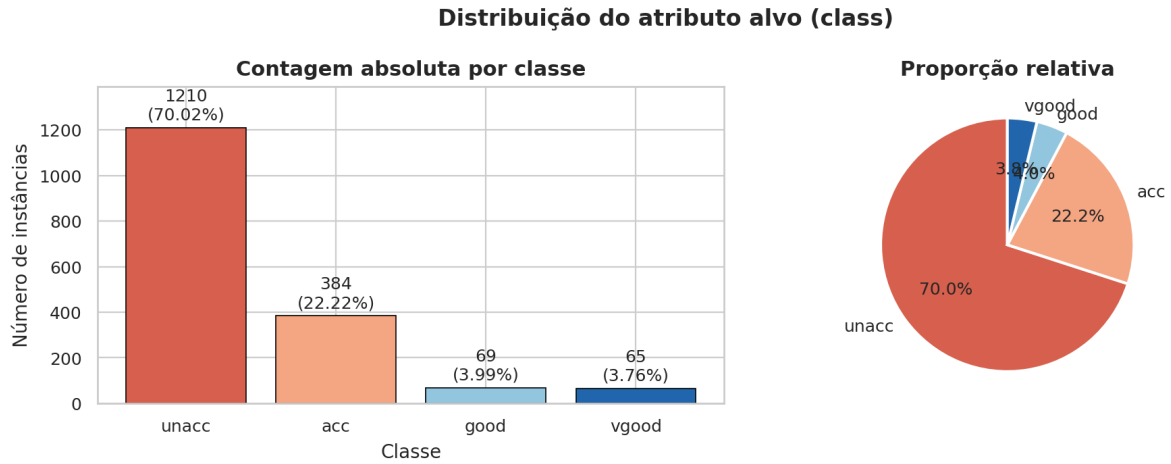


Figura 1: Distribuição do atributo alvo `class`. À esquerda, contagens absolutas; à direita, proporções relativas.

Implicações. O desbalanceamento severo torna a acurácia simples uma métrica enganosa: um classificador trivial que sempre prediga `unacc` já obteria $\approx 70\%$ de acerto. Por isso, na fase de *spot-checking*, recomendamos:

- **métrica principal:** F1 macro ou *balanced accuracy*;
- **métricas secundárias:** F1 por classe, matriz de confusão, acurácia;
- **validação cruzada estratificada**, para preservar as proporções (especialmente das classes minoritárias `good` e `vgood`).

2.3 Distribuição dos atributos preditores

A Figura 2 mostra que **todos os atributos têm distribuição marginal perfeitamente uniforme**: 432 instâncias por valor nos atributos de cardinalidade 4 e 576 instâncias por valor nos de cardinalidade 3.

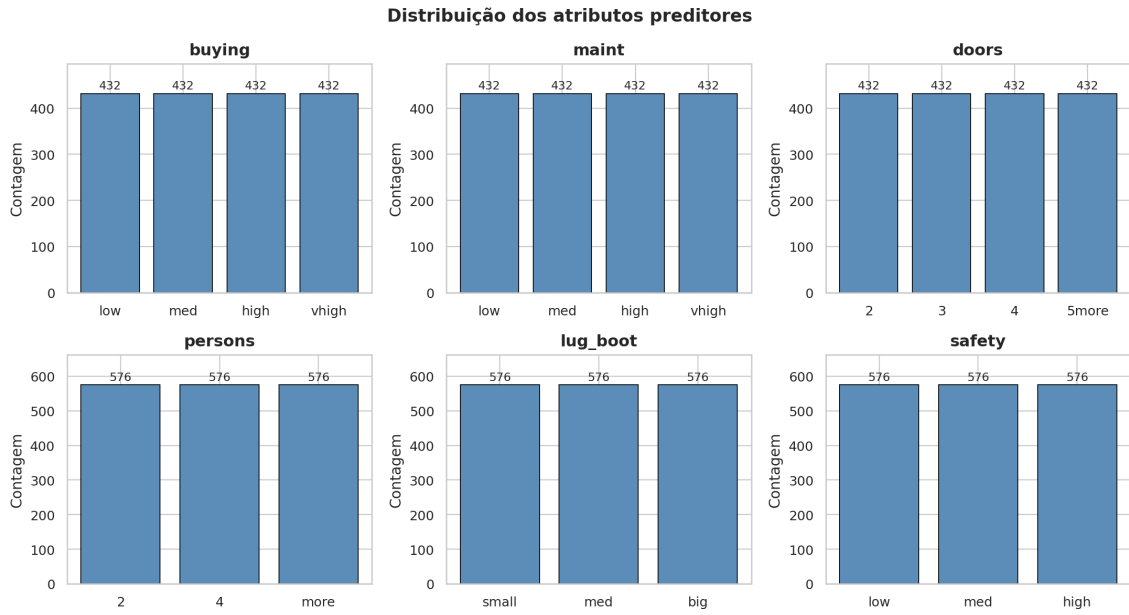


Figura 2: Distribuição marginal de cada atributo preditor. A uniformidade perfeita vem da construção do conjunto como produto cartesiano dos domínios.

2.4 Análise bivariada: classe condicionada ao atributo

A Figura 3 mostra, para cada atributo, a distribuição da classe entre os valores do atributo (proporções normalizadas por linha).

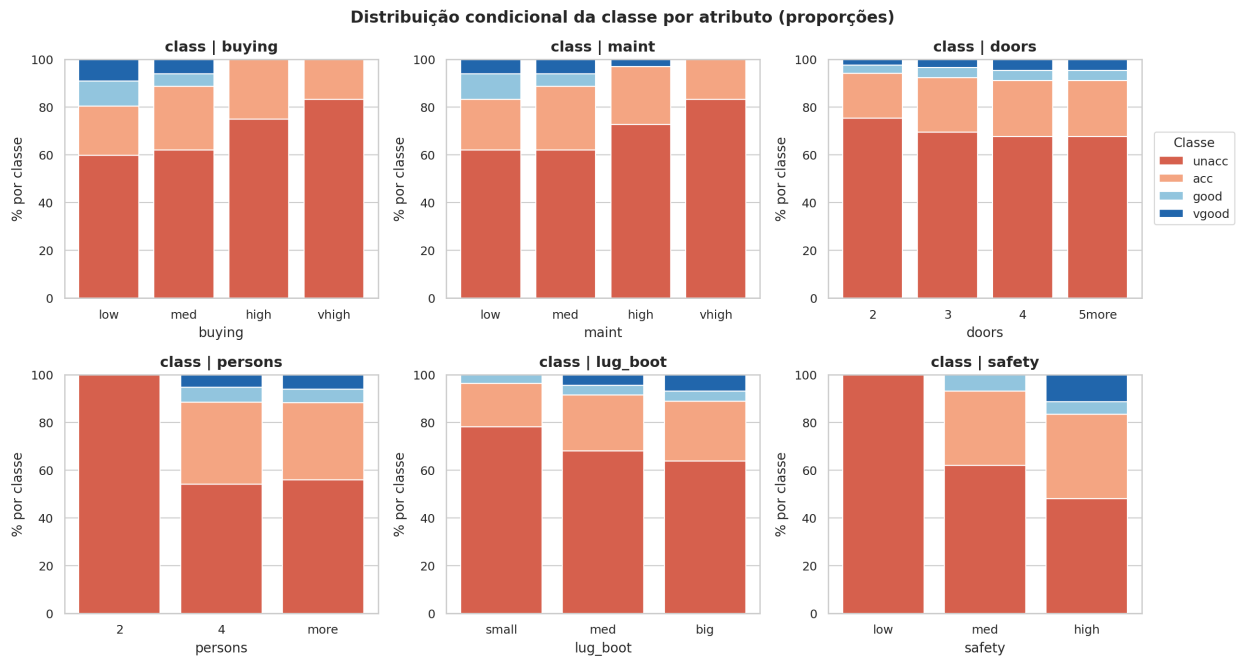


Figura 3: Distribuição condicional $P(\text{classe} \mid \text{atributo})$ para cada um dos seis atributos preditores.

Vários padrões chamam a atenção:

- **Regras determinísticas:** $\text{safety} = \text{low} \Rightarrow 100\% \text{ unacc}$ ($576/576$) e $\text{persons} = 2 \Rightarrow 100\% \text{ unacc}$ ($576/576$). Essas duas regras isoladas já explicam $960/1210 \approx 79\%$ das instâncias

inaceitáveis, e correspondem a restrições de bom senso (um carro sem segurança ou só com dois lugares dificilmente atende a uma família).

- **Restrição em `lug_boot`:** a classe `vgood` *nunca* aparece quando `lug_boot = small` ($P = 0$).
- **Tendências monotônicas:** `buying` e `maint` apresentam aumento monotônico em $P(\text{unacc})$ conforme o preço ou a manutenção sobem; `persons`, `lug_boot` e `safety` se comportam de forma contrária (aumentar o atributo melhora a avaliação).
- **Atributo aparentemente irrelevante:** `doors` produz distribuições quase idênticas em todos os seus quatro níveis.

2.5 Heatmap de probabilidades condicionais

Para reforçar visualmente os padrões discutidos, a Figura 4 mostra as matrizes $P(\text{class} \mid \text{atributo})$ na forma de *heatmap*, com os atributos em ordem natural crescente.

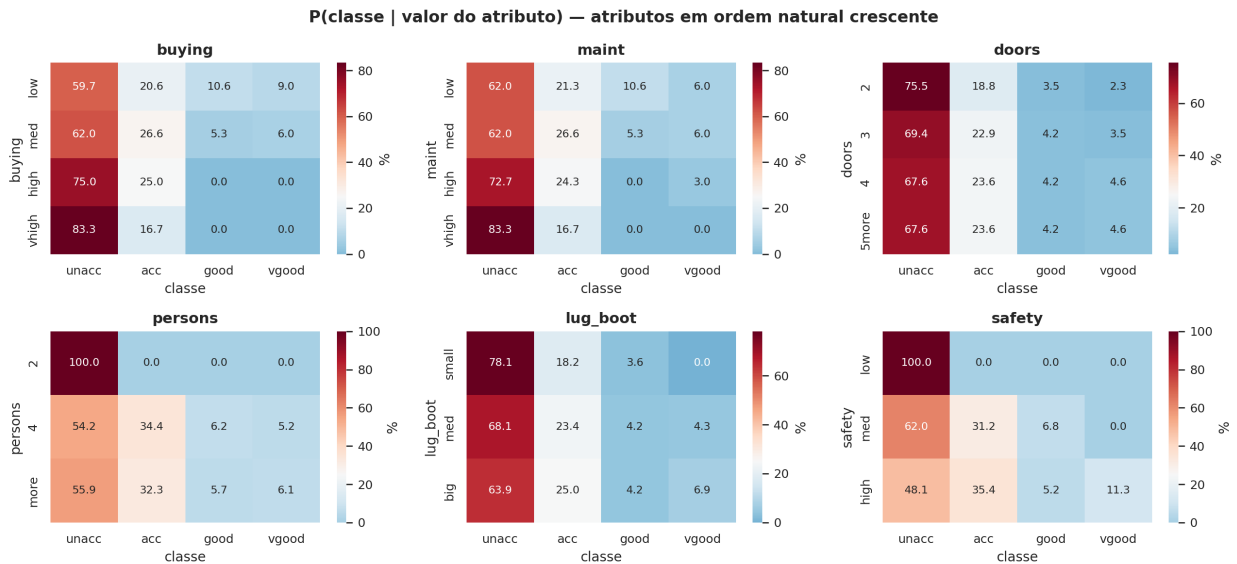


Figura 4: Probabilidade da classe condicionada ao valor de cada atributo. Tons mais escuros indicam maior concentração; as linhas dos atributos estão em ordem natural crescente, deixando claros os gradientes de monotonicidade discutidos.

2.6 Conclusões da análise exploratória

A análise nos deu uma caracterização completa do conjunto e oferece direcionamento concreto para a próxima etapa.

Características do conjunto.

1. Tamanho de 1.728 instâncias e seis atributos preditores categóricos ordinais, gerados como produto cartesiano completo — portanto, sem aleatoriedade na construção;
2. qualidade dos dados é excelente: nenhum valor faltante, nenhuma duplicata, nenhum *outlier* categórico, com todos os atributos tendo distribuição marginal perfeitamente uniforme;

- o problema é de **classificação multiclasse com classes ordenadas** e fortemente desbalanceadas (razão majoritária/minoritária de $\approx 18,6\times$).

Regras determinísticas observadas.

- `safety = low` $\Rightarrow$ 100 % unacc (576/576);
- `persons = 2` $\Rightarrow$ 100 % unacc (576/576);
- `lug_boot = small` $\Rightarrow$ nunca classifica como vgood.

Implicações para o *spot-checking*.

- Métrica principal:** F1 macro (ou *balanced accuracy*), por causa do desbalanceamento severo;
- Métricas secundárias:** acurácia, F1 por classe e matriz de confusão;
- Validação:** *stratified k-fold* para preservar as proporções;

Necessidades de pré-processamento. A EDA aponta as seguintes ações para a fase posterior do trabalho:

- codificação dos atributos categóricos — duas alternativas a considerar: *ordinal* (preserva a ordem natural identificada) e *one-hot* (trata como nominal);
- tratamento do desbalanceamento (no mínimo, estratificação na validação cruzada);
- decidir se mantemos ou descartamos `doors`, dado que as distribuições de classe são quase idênticas em todos os seus quatro níveis.

2.7 Pré-processamento dos dados

A EDA das subseções anteriores mostrou um conjunto de dados de qualidade excepcional: sem valores faltantes, sem duplicatas, com todos os atributos categóricos de baixa cardinalidade e domínios bem definidos. Como consequência, o pré-processamento necessário acaba sendo reduzido. Apresentamos as decisões a seguir de forma exaustiva, inclusive aquelas em que decidimos explicitamente *não fazer nada*, com base nos achados da Subseção 2.6.

Passos não aplicáveis. Os seguintes passos típicos de pré-processamento foram considerados e descartados:

- Imputação de valores faltantes:** desnecessária, pois o conjunto não tem valores ausentes em nenhuma das 1.728 instâncias.
- Remoção de duplicatas:** desnecessária, pois não há linhas duplicadas (coerente com a construção como produto cartesiano).

- **Remoção de *outliers*:** não se aplica, pois todos os atributos são categóricos com domínio finito; não há *outliers* no sentido estatístico clássico.
- **Normalização ou padronização:** não há atributos numéricos no conjunto original. Depois da codificação ordinal descrita logo abaixo, os valores ficam em faixas de magnitude similar ($\{0, 1, 2, 3\}$ para atributos de cardinalidade 4 e $\{0, 1, 2\}$ para os de cardinalidade 3). Padronizar a mais traria, no melhor caso, ganho marginal para os algoritmos sensíveis à escala (KNN, SVM, Regressão Logística e Rede Neural). Por isso optamos por não aplicar nesta etapa de *spot-checking*, mantendo o pipeline simples e a comparação entre algoritmos mais consistente.

Codificação dos atributos preditores. A EDA mostrou que os seis atributos têm ordem natural crescente (por exemplo, *safety*: $\text{low} \prec \text{med} \prec \text{high}$; *buying*: $\text{low} \prec \text{med} \prec \text{high} \prec \text{vhigh}$). Consideramos duas alternativas de representação:

- ***One-hot encoding*:** trataria cada nível como categoria nominal independente, expandindo os 6 atributos em 21 *features* binárias. Essa representação *descartaria* a informação de ordem identificada na EDA, e os padrões de monotonicidade observados (por exemplo, $P(\text{unacc})$ crescente com *buying* e decrescente com *safety*) ficariam mais difíceis de capturar, sobretudo para modelos lineares.
- **Codificação ordinal:** cada nível é mapeado para um inteiro respeitando a ordem natural ($\text{low} \rightarrow 0$, $\text{med} \rightarrow 1$, $\text{high} \rightarrow 2$, $\text{vhigh} \rightarrow 3$, e analogamente para os demais atributos), preservando a relação de ordem. Mantém o conjunto compacto (6 *features*) e permite que algoritmos baseados em árvore aproveitem os *splits* ordinais naturalmente.

Optamos pela **codificação ordinal**, considerando que: (i) a EDA mostrou claramente a ordem natural; (ii) os padrões monotônicos identificados se traduzem em sinal preditivo direto nessa representação; (iii) os valores resultantes ficam em faixas de magnitude similar, dispensando padronização adicional.

A implementação usou o `OrdinalEncoder` do *scikit-learn* com as ordens explicitamente especificadas para cada atributo, evitando que a codificação dependesse da ordem alfabética ou de aparição dos valores.

Codificação da variável alvo. A classe `class` foi mantida em seu formato original de *strings* (`unacc`, `acc`, `good`, `vgood`). Os classificadores que usamos na próxima etapa lidam nativamente com rótulos textuais, fazendo internamente uma codificação de rótulos. A ordem natural da classe não é explorada por nenhum dos algoritmos escolhidos — todos tratam o alvo como nominal — e foi preservada apenas para visualização coerente das matrizes de confusão e como possível direção futura de modelagem (regressão ordinal).

Decisão sobre o atributo *doors*. A EDA mostrou que *doors* apresenta distribuições de classe quase idênticas em seus quatro níveis (Figuras 3 e 4), o que sugere baixa relevância preditiva. Consideramos duas opções: (a) descartar o atributo, simplificando o espaço de *features*; (b) manter e deixar a decisão a cargo dos algoritmos.

Decidimos **manter o atributo**. As razões: o custo computacional de incluí-lo é desprezível (são só 6 atributos no total); algoritmos baseados em árvore ignoram automaticamente atributos pouco informativos via critério de impureza, e modelos lineares atribuem coeficiente próximo de zero; remover cedo poderia eliminar uma interação sutil entre `doors` e outros atributos que algum algoritmo eventualmente capture. A relevância real de `doors` poderá ser confirmada em etapas futuras, possivelmente com técnicas de interpretabilidade como análise de importância de atributos.

Tratamento do desbalanceamento. Como apontado na EDA, a base é fortemente desbalanceada ($\approx 18,6\times$ entre a classe majoritária e a minoritária). Optamos por **não aplicar técnicas de reamostragem** (SMOTE, *oversampling* ou *undersampling* aleatório) nesta etapa, por três motivos: (i) técnicas de *oversampling* sintético alteram a distribuição empírica e podem introduzir viés artificial, o que é especialmente delicado em um conjunto cujas regras geradoras são determinísticas; (ii) *undersampling* jogaria fora informação útil da classe majoritária; (iii) o impacto da reamostragem deve ser avaliado de forma controlada na etapa posterior de otimização, e não introduzido sem comparação. O desbalanceamento é mitigado nesta etapa de *spot-checking* pela combinação de **validação cruzada estratificada** (preserva as proporções em cada *fold*) com **F1-macro** como métrica principal de comparação (pondera todas as classes igualmente).

Resumo das decisões. A Tabela 2 consolida as decisões adotadas.

Tabela 2: Resumo das decisões de pré-processamento.

Etapa	Decisão	Justificativa
Valores faltantes	Nada a fazer	0/1728 instâncias com faltantes
Duplicatas	Nada a fazer	0 duplicatas
<i>Outliers</i>	Não aplicável	Atributos exclusivamente categóricos
Padronização	Não aplicada	Faixas {0, 1, 2, 3} de magnitude similar
Codificação dos atributos	Ordinal	Ordem natural identificada na EDA
Codificação do alvo	Mantida como string	Classificadores lidam nativamente; alvo tratado como nominal
Atributo <code>doors</code>	Mantido	Custo desprezível; relevância validada <i>a posteriori</i>
Desbalanceamento	Sem reamostragem	Mitigado via CV estratificada + F1-macro

3 Definição da abordagem, algoritmos e estratégia de avaliação

3.1 Abordagem

Conforme caracterizado nas seções anteriores, o problema é de **classificação multiclasse supervisionada**, com quatro rótulos ordenados e forte desbalanceamento entre classes. Usamos uma abordagem padrão de aprendizado supervisionado, comparando algoritmos com vieses indutivos distintos sob a mesma estratégia de validação.

3.2 Algoritmos selecionados

Para obter diversidade de viés indutivo, conforme orientação do enunciado, selecionamos oito algoritmos cobrindo diferentes famílias de aprendizado:

- **Baseado em instâncias:** K-Nearest Neighbors (KNN), com $k = 5$;
- **Probabilístico:** Naive Bayes;
- **Linear:** Regressão Logística;
- **Não-linear paramétrico:** SVM (kernel RBF);
- **Não-linear não-paramétrico:** Árvore de Decisão;
- **Ensembles de árvores:** Random Forest (bagging) e AdaBoost (boosting);
- **Conexionista:** Rede Neural (Multilayer Perceptron).

Os algoritmos foram usados em configuração próxima à padrão do *scikit-learn*, com pequenas exceções pontuais necessárias para evitar problemas de convergência ou *overfitting* extremo, conforme detalhado na Tabela 3. Não fizemos otimização sistemática de hiperparâmetros nesta etapa, mantendo o foco no *spot-checking* comparativo entre famílias de algoritmos.

Tabela 3: Configuração dos algoritmos. Hiperparâmetros não listados foram mantidos nos valores padrão do *scikit-learn*.

Algoritmo	Hiperparâmetros explícitos
KNN	<code>n_neighbors=5</code> (igual ao padrão)
Naive Bayes	— (padrões)
Regressão Logística	<code>max_iter=200</code> (para garantir convergência)
SVM	<code>probability=True</code> (necessário para ROC-AUC), <i>kernel</i> RBF
Árvore de Decisão	<code>max_depth=8</code> , <code>ccp_alpha=0,01</code> (controle de <i>overfitting</i>)
Random Forest	— (padrões)
AdaBoost	— (padrões; estimador base é <i>stump</i>)
Rede Neural (MLP)	<code>max_iter=2000</code> (para garantir convergência)

Vale notar que a Árvore de Decisão recebeu poda explícita via `ccp_alpha` e profundidade máxima limitada. Tomamos essa decisão depois de observar, em testes preliminares, que a árvore sem restrições tendia a memorizar o conjunto de treino (consequência natural de o conjunto ser derivado de regras determinísticas conhecidas). Com essa restrição, a comparação com o *Random Forest* fica mais informativa, pois o ensemble passa a oferecer ganho real sobre uma árvore individual já razoavelmente regularizada.

3.3 Métricas de desempenho

Calculamos sete métricas, complementares entre si:

- **F1-macro** — *métrica principal* de comparação. Calcula o F1 separadamente para cada classe e tira a média não-ponderada, atribuindo peso igual a todas as classes, independentemente de sua frequência. É adequada quando as classes minoritárias (*good*, *vgood*) importam tanto quanto a majoritária;

- **Acurácia balanceada** — métrica complementar próxima ao F1-macro, calculada como a média não-ponderada do recall por classe;
- **Acurácia** — incluída por completude, mas reportada com cautela: o classificador trivial que sempre prediga `unacc` já atinge $\approx 70\%$;
- **F1-micro** — equivalente à acurácia em problemas multiclasse com avaliação global;
- **Precisão e recall** — ambos com averaging macro;
- **ROC-AUC** — calculada na estratégia *one-vs-rest* (OvR) com averaging macro, oferecendo uma visão complementar baseada em probabilidades estimadas.

3.4 Estratégia de validação

A divisão dos dados usou **validação cruzada estratificada com $k = 5$ folds**, preservando as proporções das classes em cada partição — essencial dado o forte desbalanceamento identificado na EDA. Para avaliar a estabilidade dos resultados, repetimos o experimento com quatro sementes aleatórias distintas: {16, 42, 67, 94}.

Reprodutibilidade. Para cada semente, o mesmo objeto `StratifiedKFold` foi reutilizado por todos os oito algoritmos, garantindo que todos os modelos sejam avaliados sobre *as mesmas divisões dos dados* em cada iteração, conforme orientação explícita do enunciado.

4 Spot-checking de algoritmos

4.1 Resultados por seed

As Figuras 5–8 mostram, para cada uma das quatro sementes avaliadas, a média e o desvio-padrão das sete métricas ao longo dos cinco folds da validação cruzada estratificada.

Exibindo resultados da seed 16			Exibindo resultados da seed 67		
Reg. Log			Reg. Log		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.8256	0.0214	acurácia	0.8336	0.0293
acurácia_balanceda	0.657	0.0695	acurácia_balanceda	0.6789	0.0529
recall	0.657	0.0695	recall	0.6789	0.0529
precisão	0.7409	0.046	precisão	0.7447	0.0619
f1_micro	0.8256	0.0214	f1_micro	0.8336	0.0293
f1_macro	0.6834	0.0547	f1_macro	0.7009	0.0547
roc_auc	0.9487	0.0079	roc_auc	0.9488	0.0143

KNN			KNN		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9407	0.0174	acurácia	0.9349	0.0099
acurácia_balanceda	0.828	0.0527	acurácia_balanceda	0.803	0.0264
recall	0.828	0.0527	recall	0.803	0.0264
precisão	0.8999	0.0481	precisão	0.8907	0.0378
f1_micro	0.9407	0.0174	f1_micro	0.9349	0.0099
f1_macro	0.8555	0.0509	f1_macro	0.8361	0.028
roc_auc	0.9867	0.0057	roc_auc	0.9875	0.0027

Árvore de Decisão			Árvore de Decisão		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9103	0.016	acurácia	0.8987	0.0109
acurácia_balanceda	0.7844	0.0581	acurácia_balanceda	0.6533	0.0442
recall	0.7844	0.0581	recall	0.6533	0.0442
precisão	0.8127	0.0589	precisão	0.6574	0.1358
f1_micro	0.9103	0.016	f1_micro	0.8987	0.0109
f1_macro	0.7806	0.054	f1_macro	0.6282	0.0601
roc_auc	0.9779	0.0052	roc_auc	0.9647	0.0051

Rede Neural			Rede Neural		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9103	0.016	acurácia	0.9703	0.0198
acurácia_balanceda	0.7844	0.0581	acurácia_balanceda	0.9375	0.0378
recall	0.7844	0.0581	recall	0.9375	0.0378
precisão	0.8127	0.0589	precisão	0.9518	0.0388
f1_micro	0.9103	0.016	f1_micro	0.9703	0.0198
f1_macro	0.7806	0.054	f1_macro	0.9422	0.0332
roc_auc	0.9779	0.0052	roc_auc	0.9973	0.0025

Naive Bayes			Naive Bayes		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.6968	0.0149	acurácia	0.6997	0.025
acurácia_balanceda	0.5787	0.0262	acurácia_balanceda	0.5712	0.0326
recall	0.5787	0.0262	recall	0.5712	0.0326
precisão	0.5601	0.0681	precisão	0.5651	0.0988
f1_micro	0.6968	0.0149	f1_micro	0.6997	0.025
f1_macro	0.4516	0.0327	f1_macro	0.4447	0.0464
roc_auc	0.897	0.0101	roc_auc	0.8957	0.0112

SVM			SVM		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9537	0.0177	acurácia	0.9508	0.0111
acurácia_balanceda	0.883	0.0457	acurácia_balanceda	0.8714	0.03
recall	0.883	0.0457	recall	0.8714	0.03
precisão	0.9369	0.0322	precisão	0.919	0.0293
f1_micro	0.9537	0.0177	f1_micro	0.9508	0.0111
f1_macro	0.9016	0.0418	f1_macro	0.886	0.0169
roc_auc	0.9967	0.0019	roc_auc	0.9961	0.002

Random Forest			Random Forest		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9703	0.009	acurácia	0.9703	0.0138
acurácia_balanceda	0.927	0.0398	acurácia_balanceda	0.9326	0.0159
recall	0.927	0.0398	recall	0.9326	0.0159
precisão	0.938	0.0269	precisão	0.9486	0.0314
f1_micro	0.9703	0.009	f1_micro	0.9703	0.0138
f1_macro	0.9281	0.0301	f1_macro	0.9368	0.0247
roc_auc	0.9982	0.0009	roc_auc	0.998	0.0011

AdaBoost			AdaBoost		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.8669	0.021	acurácia	0.8466	0.0168
acurácia_balanceda	0.6324	0.0701	acurácia_balanceda	0.5745	0.0649
recall	0.6324	0.0701	recall	0.5745	0.0649
precisão	0.6987	0.1231	precisão	0.6072	0.0888
f1_micro	0.8669	0.021	f1_micro	0.8466	0.0168
f1_macro	0.6285	0.0762	f1_macro	0.5619	0.0687
roc_auc	0.9339	0.0032	roc_auc	0.9356	0.009

Figura 5: Scores obtidos com a seed 16

Figura 6: Scores obtidos com a seed 67

Exibindo resultados da seed 94			Exibindo resultados da seed 42		
Reg. Log			Reg. Log		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.8278	0.0122	acurácia	0.8234	0.0209
acurácia_balanceada	0.6526	0.042	acurácia_balanceada	0.658	0.0634
recall	0.6526	0.042	recall	0.658	0.0634
precisão	0.6953	0.0329	precisão	0.7372	0.0747
f1_micro	0.8278	0.0122	f1_micro	0.8234	0.0209
f1_macro	0.6655	0.037	f1_macro	0.676	0.0623
roc_auc	0.952	0.0067	roc_auc	0.9468	0.0057

KNN			KNN		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9276	0.0065	acurácia	0.9276	0.0065
acurácia_balanceada	0.773	0.0269	acurácia_balanceada	0.773	0.0269
recall	0.773	0.0269	recall	0.773	0.0269
precisão	0.861	0.0399	precisão	0.861	0.0399
f1_micro	0.9276	0.0065	f1_micro	0.9276	0.0065
f1_macro	0.8058	0.0258	f1_macro	0.8058	0.0258
roc_auc	0.9813	0.0062	roc_auc	0.9813	0.0062

Árvore de Decisão			Árvore de Decisão		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9096	0.0182	acurácia	0.8987	0.0312
acurácia_balanceada	0.737	0.0346	acurácia_balanceada	0.7394	0.0527
recall	0.737	0.0346	recall	0.7394	0.0527
precisão	0.8234	0.0479	precisão	0.7863	0.0574
f1_micro	0.9096	0.0182	f1_micro	0.8987	0.0312
f1_macro	0.7469	0.0291	f1_macro	0.7422	0.0484
roc_auc	0.9765	0.0034	roc_auc	0.9751	0.006

Rede Neural			Rede Neural		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9711	0.0153	acurácia	0.9711	0.006
acurácia_balanceada	0.9387	0.0232	acurácia_balanceada	0.9363	0.0164
recall	0.9387	0.0232	recall	0.9363	0.0164
precisão	0.951	0.0321	precisão	0.9628	0.0244
f1_micro	0.9711	0.0153	f1_micro	0.9711	0.006
f1_macro	0.9439	0.0267	f1_macro	0.9474	0.0158
roc_auc	0.997	0.0025	roc_auc	0.9978	0.0009

Naive Bayes			Naive Bayes		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.7142	0.0187	acurácia	0.7004	0.0172
acurácia_balanceada	0.5817	0.024	acurácia_balanceada	0.572	0.0255
recall	0.5817	0.024	recall	0.572	0.0255
precisão	0.5736	0.052	precisão	0.5462	0.0473
f1_micro	0.7142	0.0187	f1_micro	0.7004	0.0172
f1_macro	0.4612	0.034	f1_macro	0.4438	0.0272
roc_auc	0.9011	0.0059	roc_auc	0.8991	0.007

SVM			SVM		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9544	0.0084	acurácia	0.9522	0.0126
acurácia_balanceada	0.8687	0.0236	acurácia_balanceada	0.8804	0.0389
recall	0.8687	0.0236	recall	0.8804	0.0389
precisão	0.9275	0.0334	precisão	0.9263	0.0233
f1_micro	0.9544	0.0084	f1_micro	0.9522	0.0126
f1_macro	0.8934	0.022	f1_macro	0.8947	0.0189
roc_auc	0.9962	0.0023	roc_auc	0.9964	0.0022

Random Forest			Random Forest		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.9682	0.007	acurácia	0.9725	0.0029
acurácia_balanceada	0.9173	0.0477	acurácia_balanceada	0.9109	0.0411
recall	0.9173	0.0477	recall	0.9109	0.0411
precisão	0.9353	0.0118	precisão	0.9416	0.0312
f1_micro	0.9682	0.007	f1_micro	0.9725	0.0029
f1_macro	0.9232	0.0324	f1_macro	0.9236	0.0326
roc_auc	0.9981	0.0007	roc_auc	0.9985	0.0005

AdaBoost			AdaBoost		
	media	desvio_padrao		media	desvio_padrao
acurácia	0.8408	0.0109	acurácia	0.8539	0.028
acurácia_balanceada	0.6151	0.079	acurácia_balanceada	0.6204	0.0857
recall	0.6151	0.079	recall	0.6204	0.0857
precisão	0.6913	0.0918	precisão	0.6372	0.0779
f1_micro	0.8408	0.0109	f1_micro	0.8539	0.028
f1_macro	0.6225	0.0823	f1_macro	0.6119	0.0883
roc_auc	0.9348	0.0072	roc_auc	0.9357	0.0015

Figura 7: Scores obtidos com a seed 94

Figura 8: Scores obtidos com a seed 42

Os resultados foram consistentes entre as quatro *seeds*, o que indica baixa sensibilidade dos algoritmos à variação aleatória induzida pela permutação dos folds — todos os modelos mantêm seu desempenho relativo entre as repetições. Também é possível ver que a **Rede Neural** (MLP) e o **Random Forest** apresentaram os melhores desempenhos em todas as métricas relevantes para o problema, com desempenho próximo entre si, seguidos pelo **SVM**. Por outro lado, os modelos lineares (Regressão Logística), o classificador probabilístico (Naive Bayes) e o *boosting* com aprendiz fraco (AdaBoost) apresentaram desempenho bastante inferior.

4.2 Análise por métrica (boxplots)

Os boxplots a seguir permitem ver com mais detalhe a distribuição dos scores entre folds e seeds, mostrando não só a tendência central mas também a estabilidade de cada algoritmo.

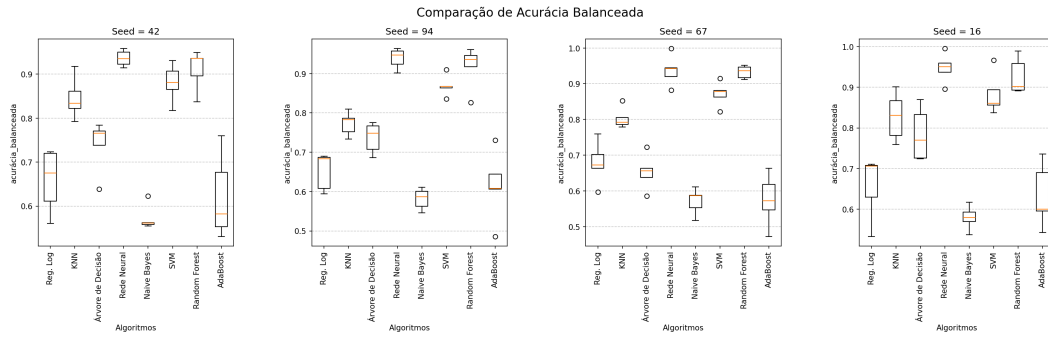


Figura 9: Boxplot da acurácia balanceada

A acurácia balanceada é uma das métricas mais informativas neste cenário, por ser robusta ao desbalanceamento. **Rede Neural** e **Random Forest** apresentam as maiores medianas (próximas de 0,93) e as menores dispersões, seguidos pelo **SVM** ($\approx 0,89$). Naive Bayes e AdaBoost, no outro extremo, têm acurácia balanceada na faixa de 0,55–0,65, mostrando dificuldade em recuperar instâncias das classes minoritárias.

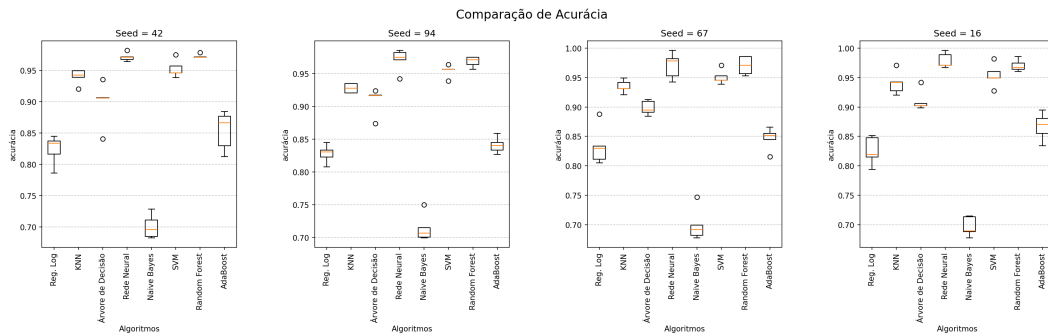


Figura 10: Boxplot da acurácia

A maioria dos algoritmos tem acurácia alta ($> 0,90$), o que é reflexo do desbalanceamento: classificar corretamente a classe majoritária unacc já basta para obter alta acurácia. Naive Bayes foge do padrão, com acurácia próxima de 0,70. A acurácia simples deve ser interpretada com cuidado neste cenário — por isso ela não é a métrica principal.

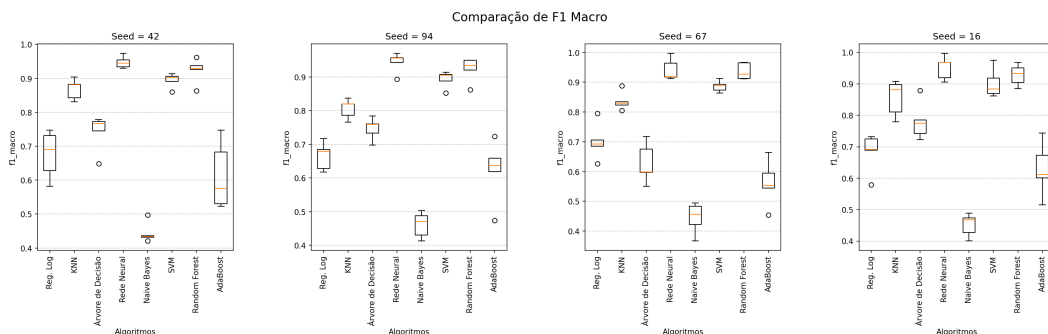


Figura 11: Boxplot do F1-macro

Esta é a métrica principal de seleção, e o gráfico revela a mudança mais significativa em relação à execução anterior: a **Rede Neural**, agora com convergência completa, lidera o ranking, com mediana ligeiramente acima do **Random Forest** (ambos próximos de 0,93–0,95). **SVM** e **KNN**

aparecem em seguida ($\approx 0,89$ e $\approx 0,84$, respectivamente). Os três algoritmos do topo têm baixa dispersão, o que indica estabilidade entre folds e *seeds*.

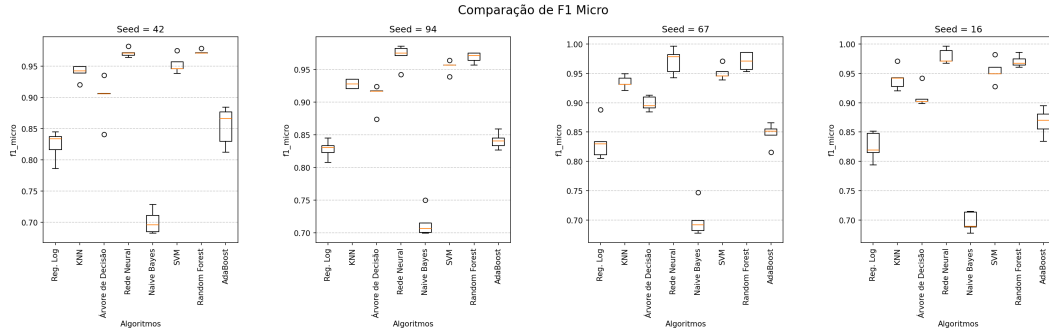


Figura 12: Boxplot do F1-micro

Em classificação multiclasse com averaging *micro*, F1-micro coincide com a acurácia, e o padrão observado é semelhante ao da Figura 10. A métrica é menos informativa neste contexto desbalanceado e foi mantida apenas por completude.

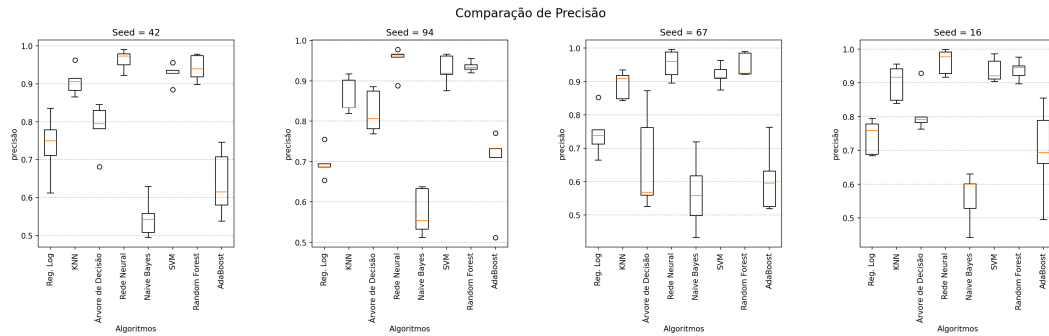


Figura 13: Boxplot da precisão (macro)

A precisão macro apresenta dispersões variadas: Rede Neural, Random Forest e SVM mantêm precisão estável e alta ($> 0,90$), enquanto Naive Bayes e AdaBoost têm medianas mais baixas e distribuições mais espalhadas, o que indica inconsistência em evitar falsos positivos.

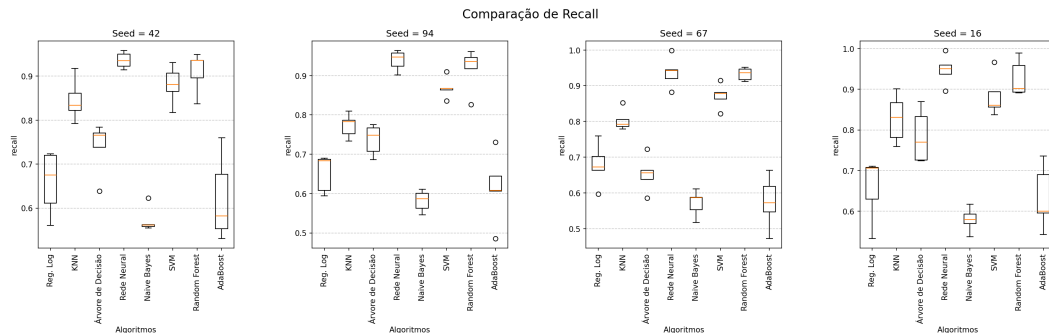


Figura 14: Boxplot do recall (macro)

O comportamento do recall macro é parecido com o da precisão. Os algoritmos com menor dispersão (Rede Neural, Random Forest, SVM) recuperam de forma consistente instâncias das classes minoritárias, enquanto AdaBoost e Naive Bayes apresentam recall baixo e variável.

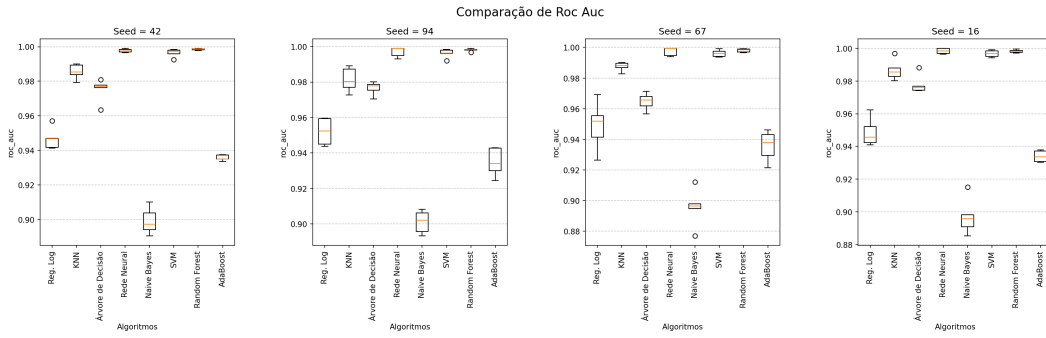


Figura 15: Boxplot do ROC-AUC (OvR macro)

O ROC-AUC confirma a qualidade dos melhores algoritmos: Random Forest, SVM e Rede Neural ficam consistentemente acima de 0,99 em várias seeds. Por se basear em probabilidades estimadas em vez do rótulo final, esta métrica é menos sensível ao desbalanceamento, mas mesmo assim mostra o *gap* entre os algoritmos do topo e o AdaBoost ($\approx 0,93$), o pior dos avaliados nesta métrica.

4.3 Análise das matrizes de confusão

As matrizes de confusão a seguir foram obtidas em um regime de avaliação diferente dos boxplots: para cada *seed*, treinamos os 8 modelos nos 80% de treino e usamos para prever os 20% reservados como *holdout*. Em seguida, concatenamos as previsões das quatro *seeds*, totalizando aproximadamente 1.384 previsões por algoritmo. Esta análise é complementar à anterior — enquanto os boxplots refletem o desempenho via validação cruzada de 5 *folds* sobre o conjunto de treino, as matrizes oferecem uma visão consolidada do comportamento dos modelos em dados não vistos durante o treino, com granularidade por classe.

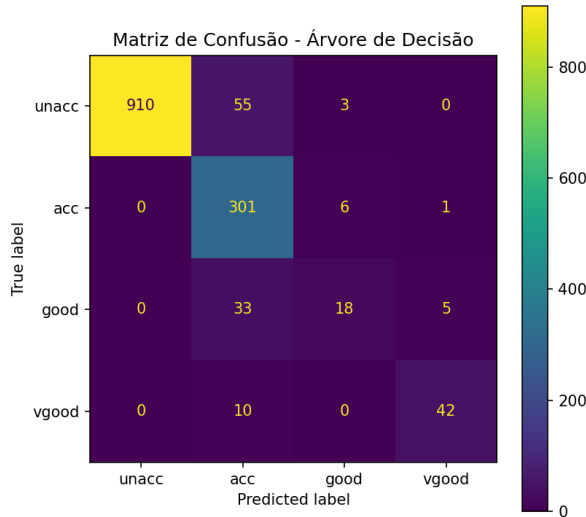


Figura 16: Matriz de confusão da Árvore de Decisão

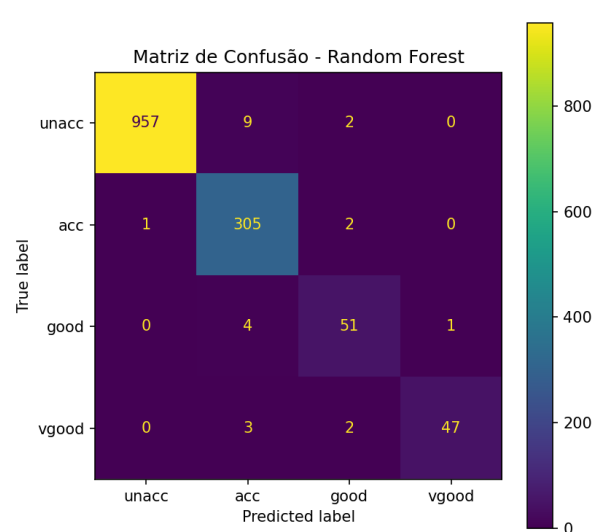


Figura 17: Matriz de confusão do Random Forest

A **Árvore de Decisão** atinge alta taxa de acerto na classe majoritária *unacc* (*recall* de 94%) e em *acc* (98%), mas tem dificuldade clara com a classe minoritária *good*: apenas 18 das 56 instâncias dessa classe são classificadas corretamente (32% de *recall*), com a maioria sendo confundida com

acc (33 instâncias). Para *vgood*, o desempenho é razoável (81%). É um comportamento típico de uma árvore sem otimização, que não consegue separar bem classes minoritárias adjacentes.

O **Random Forest** melhora bastante em todas as classes: *recall* de 99% para *unacc* e *acc*, 91% para *good* e 90% para *vgood*. A combinação de múltiplas árvores reduz a variância e captura melhor as interações de atributos necessárias para discriminar as classes minoritárias. As classes ainda relativamente mais difíceis são *good* e *vgood*, com a maioria dos erros restantes sendo confusões com a classe ordinal adjacente.

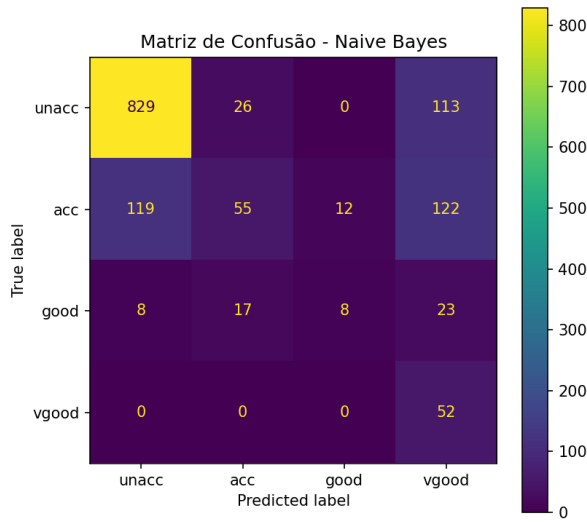


Figura 18: Matriz de confusão do Naive Bayes

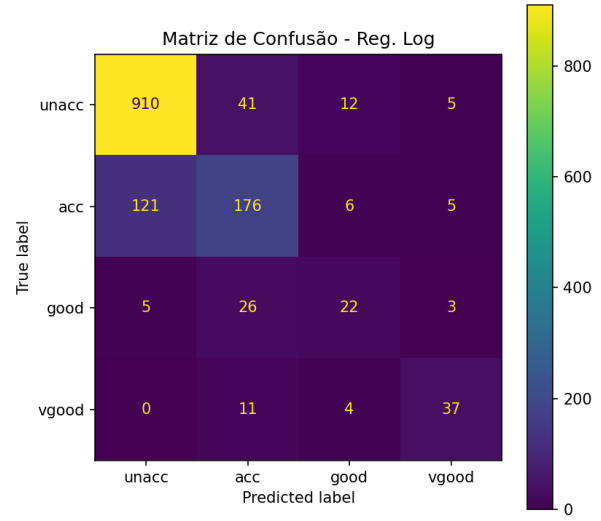


Figura 19: Matriz de confusão da Regressão Logística

O **Naive Bayes** apresenta o pior desempenho do conjunto. A hipótese de independência condicional entre atributos é fortemente violada neste problema, em que a classe depende de interações multivariadas (por exemplo, as regras determinísticas `persons = 2` ou `safety = low` $\Rightarrow$ *unacc* identificadas na Subseção 2.6). Como resultado, o modelo distribui as predições de forma anômala: 113 instâncias verdadeiramente *unacc* são classificadas como *vgood*; e das 308 instâncias *acc* no teste, apenas 55 são acertadas (18% de *recall*), com 119 falsamente atribuídas a *unacc* e 122 a *vgood*.

A **Regressão Logística** tem desempenho intermediário. Acerta 94% das instâncias *unacc*, mas sofre nas demais classes: apenas 57% de *recall* em *acc* (com 121 instâncias erroneamente classificadas como *unacc*), 39% em *good* e 71% em *vgood*. O resultado mostra que uma fronteira linear no espaço dos atributos não captura bem as interações multivariadas que o problema exige.

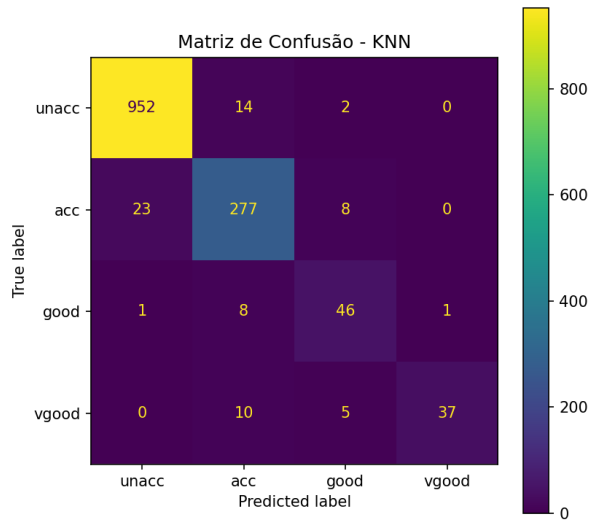
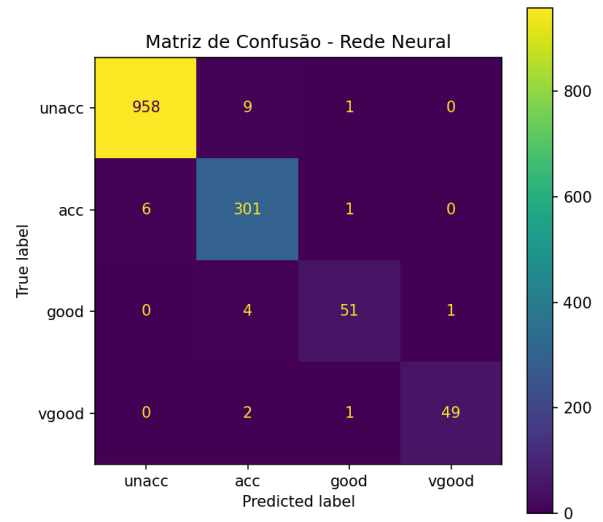
Figura 20: Matriz de confusão do KNN ($k = 5$)

Figura 21: Matriz de confusão da Rede Neural

O **KNN** apresenta desempenho consistente, com *recall* acima de 80% nas três classes mais frequentes (*unacc* 98%, *acc* 90%, *good* 82%). A classe mais problemática é *vgood* (*recall* de 71%): das 52 instâncias *vgood*, 10 acabam classificadas como *acc* e 5 como *good*, o que sugere que a métrica de distância padrão tem dificuldade em isolar as instâncias com características muito específicas das classes minoritárias.

A **Rede Neural** apresenta desempenho ótimo e equilibrado depois da convergência completa (*max_iter*=2000): *recall* de 99% em *unacc*, 98% em *acc*, 91% em *good* e 94% em *vgood*. As poucas confusões que sobram ocorrem em sua maioria entre classes ordinais adjacentes — por exemplo, 4 das 56 instâncias *good* são classificadas como *acc*, e 2 das 52 *vgood* também como *acc*. O MLP é, junto com o Random Forest, um dos dois algoritmos com desempenho mais equilibrado entre as quatro classes, confirmando empiricamente a capacidade de redes neurais de capturar as interações multivariadas do problema (regras DEX) quando há iterações suficientes para convergir.

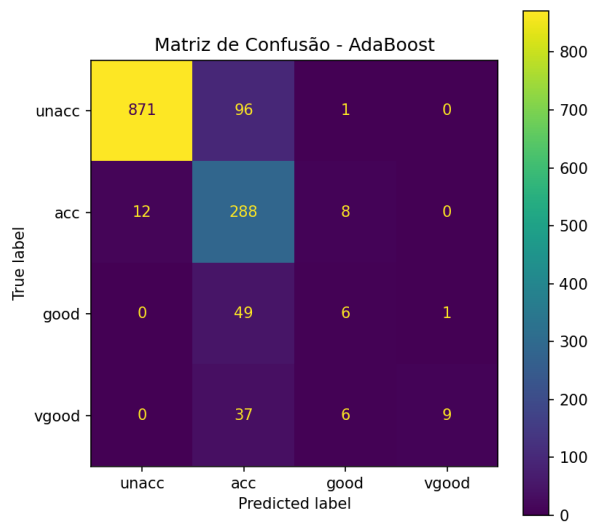


Figura 22: Matriz de confusão do AdaBoost

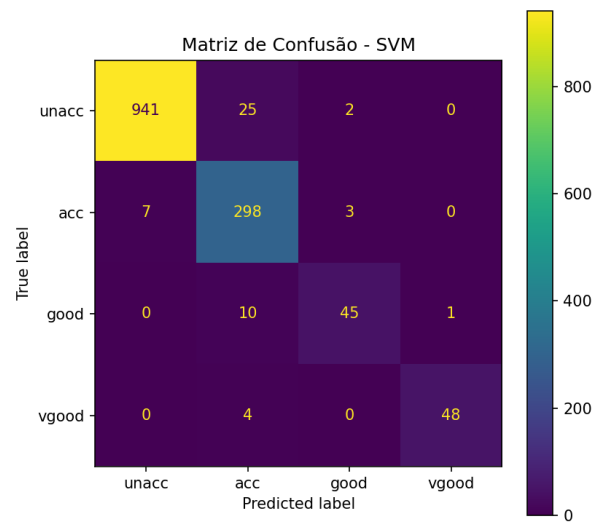


Figura 23: Matriz de confusão do SVM

O **AdaBoost** apresenta o segundo pior desempenho. O modelo concentra suas previsões na classe

acc: 96 instâncias **unacc**, 49 **good** e 37 **vgood** são erroneamente classificadas como **acc**. O resultado vem da combinação entre a estratégia de *boosting* com aprendiz fraco (*stumps* por padrão) e a estrutura hierárquica do problema, que exige interações de múltiplos atributos para discriminar bem as classes minoritárias — algo que uma sequência de *stumps* tem dificuldade em capturar.

O **SVM** apresenta o melhor desempenho entre os classificadores não baseados em árvore: *recall* acima de 80% em todas as classes (**unacc** 97%, **acc** 97%, **good** 80%, **vgood** 92%). A maior fonte de erro é a confusão entre **unacc** e **acc** (25 instâncias **unacc** previstas como **acc** e 7 no sentido inverso), padrão esperado pela proximidade dessas classes na escala de aceitabilidade.

4.4 Conclusões e algoritmos promissores

A análise comparativa entre os oito algoritmos avaliados, sob a métrica principal F1-macro (Tabela 4), permite uma seleção informada para a próxima etapa do trabalho (otimização de hiperparâmetros).

Tabela 4: F1-macro médio entre as quatro *seeds* avaliadas, ordenado do melhor para o pior.

Algoritmo	F1-macro (média)
Rede Neural	0,95
Random Forest	0,93
SVM	0,89
KNN	0,84
Árvore de Decisão	0,72
Regressão Logística	0,68
AdaBoost	0,61
Naive Bayes	0,45

Algoritmos mais promissores. Identificamos três candidatos para a fase de otimização de hiperparâmetros:

1. **Rede Neural** (F1-macro $\approx 0,95$): melhor desempenho geral depois da convergência completa (com `max_iter=2000`), com *recall* acima de 91% em todas as classes e baixa dispersão entre folds e *seeds*. A combinação de camadas ocultas com função de ativação não-linear permite capturar as interações multivariadas que as regras DEX (que geraram o conjunto) exigem. Justifica investigação adicional via ajuste sistemático da arquitetura (número e tamanho de camadas, função de ativação, regularização, taxa de aprendizado e número de iterações).
2. **Random Forest** (F1-macro $\approx 0,93$): segundo melhor, com desempenho praticamente equivalente ao da Rede Neural e baixa dispersão entre folds e *seeds*. A natureza de *ensemble* e a capacidade de capturar interações sucessivas de atributos via *splits* são particularmente adequadas ao problema, que foi originalmente derivado de regras hierárquicas (modelo DEX) naturalmente favoráveis a árvores. Representa também um viés indutivo complementar ao da Rede Neural, oferecendo alternativa robusta para a fase de otimização.
3. **SVM** (F1-macro $\approx 0,89$): terceiro melhor, com bom balanceamento entre classes e baixa variância. Representa um viés indutivo diferente (margem máxima em espaço transformado

pelo *kernel*), oferecendo uma terceira família distinta para diversificação na fase de otimização.

Algoritmos descartados. Dois algoritmos não devem seguir para a próxima etapa:

- **Naive Bayes** (F1-macro 0,450): a hipótese de independência condicional entre atributos é fundamentalmente incompatível com a estrutura do problema. A otimização de hiperparâmetros não vai corrigir essa limitação;
- **AdaBoost com configuração padrão** (F1-macro 0,607): o aprendiz fraco baseado em *stumps* não consegue capturar as interações multivariadas exigidas. Poderia ser melhorado com a troca do estimador base, mas optamos por focar em famílias com *baseline* superior.

KNN, Árvore de Decisão isolada e Regressão Logística ficam em situação intermediária; podem eventualmente ser reavaliados se houver tempo na fase seguinte, mas não constituem prioridade.

5 Otimização de Hiperparâmetros

5.1 Explicação e Ferramentas

O desempenho da maioria dos algoritmos de *machine learning* depende da escolha de hiperparâmetros usados para a construção dos modelos que os implementam. Isso também se aplica aos algoritmos mais promissores encontrados durante o *spot-checking*. Portanto, para que possamos comparar os melhores desempenhos dos modelos dos algoritmos promissores para o conjunto de dados escolhido, de forma que realmente seja escolhido o melhor modelo possível para classificar o conjunto de dados, é necessário treinar e testar o desempenho dos modelos dos algoritmos com diversos hiperparâmetros antes de realizar a comparação final de desempenho.

Para aumentar a capacidade exploratória de hiperparâmetros, testando o desempenho da maior quantidade de conjuntos de hiperparâmetros por modelo em uma quantidade razoável de tempo, o processo de otimização foi realizado com auxílio da biblioteca de otimização de hiperparâmetros **Optuna**.

A biblioteca *Optuna* usa técnicas heurísticas de busca baseadas no histórico de desempenhos alcançados com diferentes conjuntos de hiperparâmetros para, dinamicamente, selecionar conjuntos de hiperparâmetros que possuem maior probabilidade de resultar em um desempenho bom no modelo a ser testado enquanto ignora conjuntos que provavelmente resultarão em um desempenho baixo. Além disso, ela permite que o espaço de exploração dos hiperparâmetros para cada hiperparâmetro seja definido de forma concisa, abstrata, dinâmica e eficiente com recursos de memória por meio da utilização de estruturas de controle da linguagem *Python* (*if/else*, *loops*) ao invés de precisar de uma lista estática e guardada em memória com todos os valores específicos a serem testados. Dessa forma, a probabilidade de que os conjuntos de hiperparâmetros finais encontrados para os modelos finais de cada algoritmo para a classificação do conjunto de dados escolhido sejam os melhores possíveis é muito maior do que se fosse realizada apenas uma busca exaustiva com *GridSearchCV* ou uma busca aleatória com *RandomizedSearchCV* em um espaço exploratório completamente pré-definido.

5.2 Hiperparâmetros Explorados por Algoritmo e seus Espaços de Exploração

A implementação dos algoritmos analisados foi feita utilizando-se as implementações da biblioteca *scikit-learn*. Especificamente:

- **MLP (Rede Neural) $\Rightarrow$ MLPClassifier**
- **SVM $\Rightarrow$ SVC**
- **Random Forest $\Rightarrow$ RandomForestClassifier**

Dessa forma, os hiperparâmetros que podem ser otimizados e o espaço do domínio que pode ser explorado para a otimização dependem dos parâmetros dessas funções.

Abaixo, para cada uma das funções de implementação, segue uma tabela descrevendo cada hiperparâmetro explorado, explicando o que esse hiperparâmetro controla no modelo, mostrando o domínio dentro do qual o valor de cada hiperparâmetro pode variar nessas implementações, e o quanto desse domínio foi explorado na otimização.

Tabela 5: Hiperparâmetros a serem otimizados do MLPClassifier.

Hiperparâmetro	Função	Domínio	Espaço explorado
n_ layers	Número de camadas ocultas	Inteiros positivos	{1,2}
hidden_ layer_ sizes	Quantidade de neurônios em cada camada oculta	Inteiros positivos	10–200
activation	Função de ativação dos neurônios	identity, logistic, tanh, relu	Todos
solver	Algoritmo de otimização dos pesos	lbfgs, sgd, adam	Todos
alpha	Coeficiente de regularização L2	> 0	10^{-6} – 10^{-1}
batch_ size	Tamanho do mini-batch	auto ou inteiro positivo	auto, 32–512 (variando em potências de 2)
learning_ rate	Estratégia de atualização da taxa de aprendizado	constant, invscaling, adaptive	Todos
learning_ rate_ init	Taxa inicial de aprendizado	> 0	10^{-4} – 10^{-1}
power_ t	Expoente utilizado pelo método invscaling	$[0, \infty)$	0.0–0.5
shuffle	Embaralhamento das amostras	True ou False	Todos
tol	Tolerância para critério de parada	> 0	10^{-6} – 10^{-3}
warm_ start	Reutilização da solução anterior	True ou False	Todos
momentum	Fator de momentum do SGD	$[0, 1]$	0.0–1.0
nesterovs_ momentum	Uso de momentum de Nesterov	True ou False	Todos
early_ stopping	Parada antecipada baseada em validação	True ou False	Todos
validation_ fraction	Fração dos dados reservada para validação	$(0, 1)$	0.1–0.9
beta_ 1	Decaimento do primeiro momento (Adam)	$(0, 1)$	0.1–0.9, 0.999
beta_ 2	Decaimento do segundo momento (Adam)	$(0, 1)$	0.1–0.9, 0.999, 0.999999
epsilon	Constante de estabilidade numérica	> 0	10^{-9} – 10^{-1}

Note que o hiperparâmetro `n_layers` não é um parâmetro real da função `MLPClassifier`, mas sim um meta-parâmetro que define quantos valores de `hidden_layer_sizes` vamos passar para a função, gerando `n_layers` camadas com quantidade variável de neurônios (uso da biblioteca `Optuna` permite otimizar o seu valor sem que tenha de ser passado para a função `MLPClassifier`).

Além disso, os hiperparâmetros `max_iter`, `n_iter_no_change` e `max_fun` não foram explorados, com `max_iter` sendo fixado em **2000** (para garantir convergência), enquanto `n_iter_no_change` e `max_fun` foram deixados em seus valores padrão pois eles também possuem função de controlar o quanto esperamos até considerar que houve convergência, o que já é coberto pelo valor alto de `max_iter`

Tabela 6: Hiperparâmetros a serem otimizados do SVC.

Hiperparâmetro	Função	Domínio	Espaço explorado
C	Parâmetro de penalização dos erros	$C > 0$	0.1–2.0
kernel	Função kernel utilizada pela SVM	linear, poly, rbf, sigmoid	Todos
degree	Grau do kernel polinomial	Inteiros positivos	2–10
gamma	Coefficiente do kernel	scale, auto ou real positivo	scale, auto
coef0	Termo independente dos kernels poly e sigmoid	$\mathbb{R}$	0.0–1.0
shrinking	Uso da heurística shrinking	True ou False	Todos
tol	Tolerância para critério de parada	> 0	10^{-6} – 10^0
class_weight	Pesos das classes	None, balanced ou dicionário	None, balanced
decision_function_shape	Estratégia multiclasse	ovo ou ovr	Todos
break_ties	Desempate usando função de decisão	True ou False	Todos

Tabela 7: Hiperparâmetros a serem otimizados do RandomForestClassifier.

Hiperparâmetro	Função	Domínio	Espaço explorado
n_ estimators	Número de árvores da floresta	Inteiros positivos	10–200 (10 a 10)
criterion	Critério de divisão dos nós	gini, entropy, log_loss	gini, entropy
max_ depth	Profundidade máxima das árvores	None ou inteiro positivo	1–10
min_ samples_ split	Mínimo de amostras para dividir um nó	Inteiro ≥ 2 ou fração	2–20
min_ samples_ leaf	Mínimo de amostras em uma folha	Inteiro ≥ 1 ou fração	1–20
min_ weight_ fraction_ leaf	Fração mínima do peso total em uma folha	$[0, 0.5]$	0.0–0.5
max_ features	Número de atributos considerados por divisão	sqrt, log2, None ou numérico	sqrt, log2, None
max_ leaf_ nodes	Número máximo de folhas	None ou inteiro positivo	4–80 (4 a 4)
min_ impurity_ decrease	Redução mínima de impureza para dividir um nó	≥ 0	0.0–0.5
warm_ start	Reutilização da floresta anterior	True ou False	Todos
class_ weight	Pesos das classes	None, balanced, balanced_subsample ou dicionário	None, balanced, balanced_subsample
ccp_ alpha	Parâmetro de poda por complexidade de custo	≥ 0	0.0–0.9
max_ samples	Fração de amostras usada por árvore	$(0, 1]$ ou inteiro	0.1–1.0

Nota-se que o hiperparâmetro *criterion* não explora todo o espaço, mas isso é só porque *entropy* e *log_loss* são, internamente, o mesmo critério.

Normalização do SVC. O SVM com kernel não-linear é sensível à escala dos atributos. Para garantir a convergência do otimizador, o **SVC** foi envolvido em um *Pipeline* do *scikit-learn* com um **StandardScaler** aplicado antes do treinamento. O *scaler* é ajustado exclusivamente nos dados de treino de cada *fold* e aplicado por transformação nos dados de validação e teste, evitando qualquer vazamento de informação (*data leakage*).

5.3 Etapas da Metodologia da Otimização

Holdout Inicial. Inicialmente, o conjunto de dados foi dividido de forma estratificada com o método *holdout* em 2 subconjuntos:

- **treino**, contendo 80% dos dados, será usado para a otimização dos hiperparâmetros e para treinamento dos modelos finais.
- **teste**, contendo 20% dos dados, será usado para avaliação e comparação de desempenho dos modelos finais na seção seguinte.

A fins de reprodutibilidade dos resultados, essa divisão, e todas as outras operações que possuem algum estado aleatório nessa seção e na próxima, foram executadas com *seed* = 42.

Nested Cross-Validation. Dentro do conjunto de treino, criamos um 5-*fold* externo e um 3-*fold* interno. A cada laço externo, separamos o conjunto de treino em 4 *fold* de treino e 1 *fold* de teste, todos estratificados e com mesma quantidade de dados.

Dentro dos *fold* de teste, ocorre o laço interno. No laço interno, separamos o conjunto em 2 *fold* de treino, e 1 *fold* de validação, todos estratificados e com mesma quantidade de dados. Os dois *fold* de treino serão usados para treinar *n* modelos com conjuntos de hiperparâmetros distintos, e o *fold* de validação será usado para validar os modelos, estimando o desempenho de cada conjunto de hiperparâmetros. Para cada algoritmo, a quantidade de conjuntos de hiperparâmetros validadas por laço interno são:

- **MLPClassifier = 100**
- **SVC = 200**
- **RandomForestClassifier = 500**

A quantidade de conjuntos de hiperparâmetros validada para cada algoritmo foi decidida empiricamente, considerando velocidade de convergência para um desempenho bom e tempo médio de treinamento do modelo. Considerando esses valores e o total de *fold*, tanto internos quanto externos, a quantidade total de modelos treinados e validados com conjuntos de hiperparâmetros diferentes para cada algoritmo é:

- **n MLPClassifier models = $100 * 3 * 5 = 1500$**
- **n SVC models = $200 * 3 * 5 = 3000$**
- **n RandomForestClassifier models = $500 * 3 * 5 = 7500$**

Ou seja, foi avaliado uma grande quantidade de possíveis conjuntos de hiperparâmetros, de tal forma que temos uma alta probabilidade de achar conjuntos que resultem em alto desempenho para cada algoritmo.

Terminado o laço interno, o conjunto de hiperparâmetros que ter resultado no modelo de maior desempenho no laço interno terá sido salvo e recuperado no laço externo. No laço externo, será construído um modelo para cada algoritmo com esses hiperparâmetros. Esses modelos serão treinados nos 4 *fold* de treino externos e avaliados no 1 *fold* de teste externo. Essa avaliação é a estimação do desempenho do modelo com, teoricamente, seus melhores hiperparâmetros para nossa principal métrica, e não a estimação da qualidade de seus hiperparâmetros. Os hiperparâmetros que resultarem no modelo com melhor desempenho, para cada algoritmo, serão salvos para construção dos nossos modelos finais, que serão treinados no conjunto *holdout* de treino e avaliados no conjunto *holdout* de teste para análise mais aprofundada de todas as nossas métricas secundárias na próxima seção.

(Nota-se que, tanto no laço externo quanto no interno, a métrica usada para a avaliação dos modelos é o **f1_score_macro**, que é a nossa principal métrica escolhida, devido ao desbalanceamento de dados do *dataset*)

5.4 Resultados da Otimização

Terminada a otimização de hiperparâmetros, os melhores desempenhos de *f1_score* para cada algoritmo foram alcançados com os seguintes hiperparâmetros:

Tabela 8: Melhores hiperparâmetros encontrados para o MLPClassifier (F1 = 1.0000).

Hiperparâmetro	Melhor valor encontrado
n_layers	2
hidden_layer_sizes	[160, 190]
activation	tanh
solver	lbfgs
alpha	10^{-2}
batch_size	512
learning_rate	constant
learning_rate_init	10^{-3}
power_t	0.5
shuffle	True
tol	10^{-6}
warm_start	True
momentum (irrelevante)	0.3
nesterovs_momentum (irrelevante)	False
early_stopping	True
validation_fraction	0.3
beta1 (irrelevante)	0.2
beta2 (irrelevante)	0.6
epsilon (irrelevante)	10^{-4}

Tabela 9: Melhores hiperparâmetros encontrados para o SVC (F1 = 0.9813).

Hiperparâmetro	Melhor valor encontrado
C	1.9
kernel	poly
degree	4
gamma	scale
coef0	1.0
shrinking	False
tol_exponent	10^{-5}
class_weight	balanced
decision_function_shape	ovr
break_ties	False

Tabela 10: Melhores hiperparâmetros encontrados para o RandomForestClassifier ($F1 = 0.9170$).

Hiperparâmetro	Melhor valor encontrado
n_estimators	140
criterion	entropy
max_depth	10
min_samples_split	7
min_samples_leaf	2
min_weight_fraction_leaf	0.0
max_features	sqrt
max_leaf_nodes	68
min_impurity_decrease	0.0
warm_start	False
class_weight	balanced_subsample
ccp_alpha	0.0
max_samples	0.9

Nota-se que, após a otimização, a **Rede Neural** e a **SVM** alcançaram desempenhos bastante impressionantes, com a SVM melhorando de $\approx 0,89$ para $\approx 0,98$, e a Rede Neural alcançando *f1_score* de 1.0, ou seja, acertando incríveis **100% das previsões!**

Nota-se também que o score da **Random Forest** na CV aninhada ($\approx 0,92$) ficou abaixo do valor observado no *spot-checking* ($\approx 0,93$). Esse comportamento é esperado: a CV aninhada produz estimativas mais conservadoras do que o *spot-checking*, pois cada modelo é avaliado em subconjuntos menores e mais variados dos dados. A avaliação definitiva de desempenho, realizada sobre o conjunto de *holdout* de teste na seção seguinte, é a medida mais confiável.

6 Comparação de Desempenho em Dados de Teste

Com os melhores hiperparâmetros encontrados na etapa anterior, cada modelo é treinado no conjunto de treino completo (80%, 1.382 instâncias) e avaliado no conjunto de teste reservado desde o início (20%, 346 instâncias) — dados que não participaram de nenhuma etapa de otimização.

Um *holdout* único introduz, em princípio, alguma variância na estimativa de desempenho: com outra divisão aleatória, as instâncias de teste seriam diferentes e os números poderiam variar levemente. Uma alternativa mais robusta seria repetir o split com múltiplas sementes e reportar a média. No entanto, essa preocupação não se aplica ao dataset Car Evaluation: por ser gerado deterministicamente por regras DEX, qualquer divisão estratificada preserva a mesma distribuição de regras no conjunto de teste, e um modelo que aprendeu a função exata generalizaria igualmente bem para qualquer split. O fato de o MLP atingir 100% confirma empiricamente esse ponto. A avaliação via *holdout* é também uma das estratégias válidas indicadas pelo enunciado, sendo complementar à validação cruzada aninhada já utilizada na otimização.

6.1 Métricas gerais

Tabela 11: Desempenho dos três modelos no conjunto de teste.

Modelo	F1-macro	Bal. Acc.	Accuracy	Precision	Recall	ROC-AUC
MLP	1,0000	1,0000	1,0000	1,0000	1,0000	1,0000
SVC	0,9875	0,9938	0,9827	0,9819	0,9938	0,9998
Random Forest	0,9566	0,9841	0,9740	0,9321	0,9841	0,9984

O MLP acerta todas as 346 instâncias do conjunto de teste — F1-macro perfeito em todas as métricas. A SVC erra apenas 6 (1,7%) e o Random Forest erra 9 (2,6%). Os três modelos têm desempenho muito acima do que foi observado no *spot-checking* (Seção 4), confirmando que a otimização de hiperparâmetros valeu a pena.

Um ponto que pode causar estranheza é o MLP atingir 100% no teste. Isso não é *overfitting*: o dataset Car Evaluation foi gerado deterministicamente por regras de um sistema especialista (modelo DEX) — não há ruído. Um modelo suficientemente expressivo com bons hiperparâmetros pode aprender a função exata, e o conjunto de teste confirma que o MLP *generalizou* corretamente.

A Figura 24 compara os modelos visualmente. Usamos um gráfico de linhas em vez de barras porque, com valores todos próximos de 1,0, as barras ficam praticamente iguais em altura — no gráfico de linhas com eixo y iniciando em 0,88 as diferenças ficam muito mais claras.

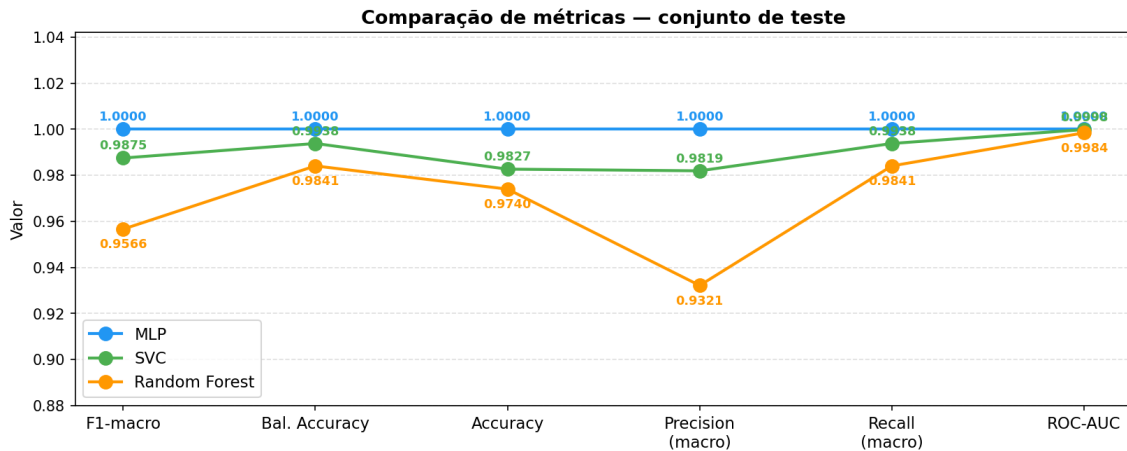


Figura 24: Comparação de métricas no conjunto de teste. Eixo y a partir de 0,88 para evidenciar as diferenças entre modelos.

6.2 Matrizes de confusão

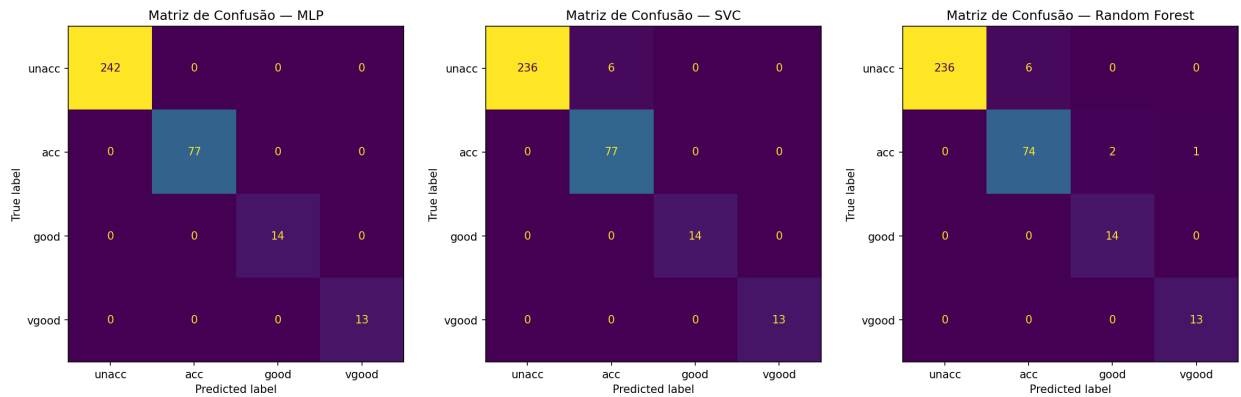


Figura 25: Matrizes de confusão dos três modelos finais no conjunto de teste.

O MLP não comete nenhum erro. A SVC erra 6 instâncias: todas da classe `unacc` classificadas como `acc`. O Random Forest erra 9: os mesmos 6 do padrão `unacc`→`acc`, mais 2 confusões `acc`→`good` e 1 `acc`→`vgood`. Em ambos os casos, os erros sempre ocorrem entre classes ordinalmente adjacentes — nunca um salto de `unacc` direto para `good` ou `vgood`, por exemplo. Isso mostra que os modelos respeitam a hierarquia entre classes mesmo sem ter sido ensinados a isso diretamente.

6.3 Curvas ROC e Precision-Recall

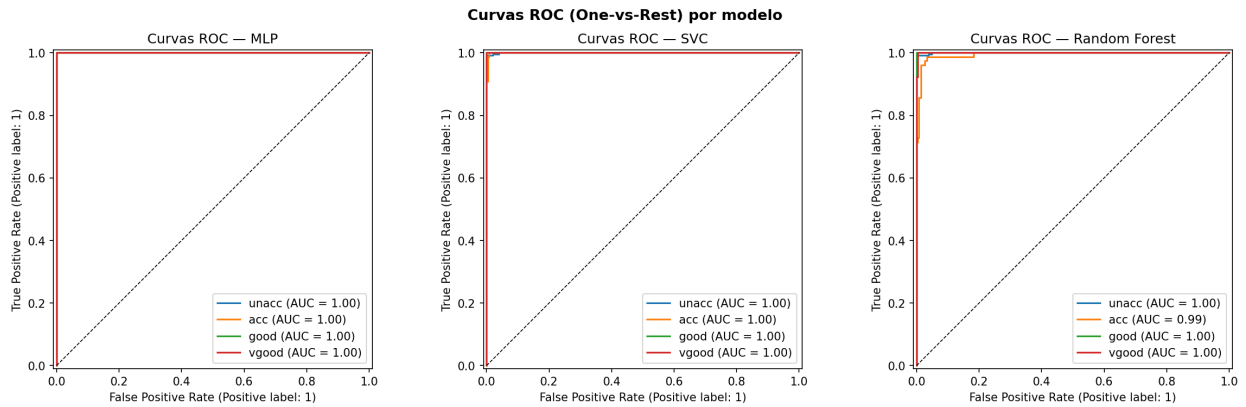


Figura 26: Curvas ROC (One-vs-Rest) por classe para cada modelo.

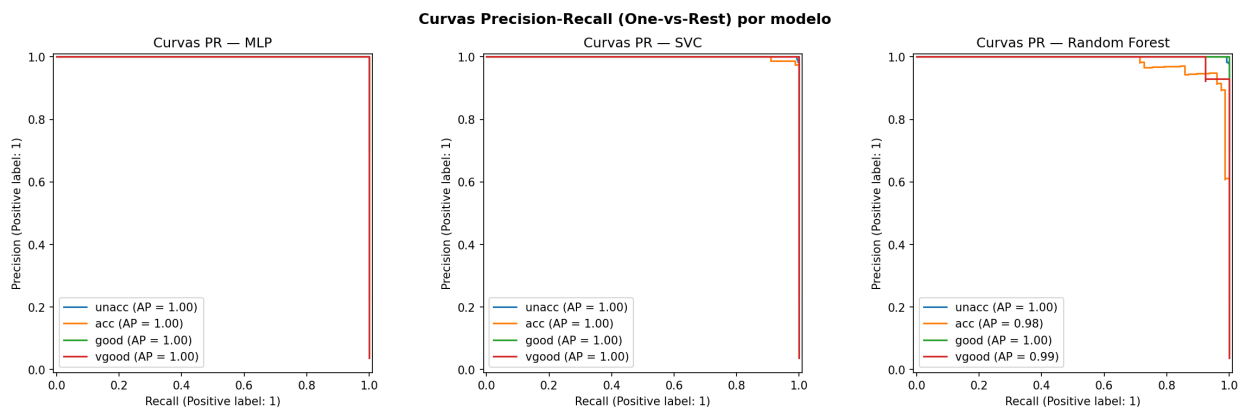


Figura 27: Curvas Precision-Recall (One-vs-Rest) por classe para cada modelo.

As curvas ROC (Figura 26) confirmam a excelência dos três modelos: AUC de 1,0000 para o MLP, 0,9998 para a SVC e 0,9984 para o Random Forest — mesmo o pior resultado está muito próximo do ideal. As curvas PR (Figura 27) mostram um leve afastamento do ideal para o Random Forest na classe *good*, que é a menos representada ($\approx 4\%$ do dataset), mas ainda assim com área muito alta.

6.4 Análise de erros, pontos fortes e fracos

Tabela 12: Erros cometidos no conjunto de teste (346 instâncias).

Modelo	Erros	Taxa	Padrão de confusão
MLP	0	0,0%	nenhum
SVC	6	1,7%	$6 \times \text{unacc} \rightarrow \text{acc}$
Random Forest	9	2,6%	$6 \times \text{unacc} \rightarrow \text{acc}$, $2 \times \text{acc} \rightarrow \text{good}$, $1 \times \text{acc} \rightarrow \text{vgood}$

MLP. Acerta todas as 346 instâncias sem exceção — não há classe mais difícil nem caso mal classificado. Todas as quatro classes têm $F1 = 1,0$ e $\text{recall} = 1,0$ individualmente. Isso é possível porque o dataset é determinístico (regras DEX, sem ruído) e o MLP com os hiperparâmetros encontrados conseguiu aprender a função exata. É claramente o modelo mais forte do experimento em todos os aspectos.

SVC. Ponto forte: altíssima confiança probabilística em todas as classes (ROC-AUC de 0,9998), recall equilibrado entre as quatro classes. **Ponto fraco:** 6 instâncias de *unacc* classificadas como *acc* — a fronteira *unacc/acc* é a única região de dificuldade. Essa é a classe mais difícil para a SVC não pelo número absoluto de erros, mas pela posição: *unacc* e *acc* são as classes mais frequentes e ordinalmente vizinhas, compartilhando combinações de atributos intermediários (por exemplo, segurança média com custo moderado).

Random Forest. Ponto forte: recall alto (0,9841), bom desempenho geral. **Ponto fraco:** precisão macro mais baixa (0,9321), indicando alguns falsos positivos nas classes minoritárias. Os 9 erros se distribuem por duas fronteiras: *unacc/acc* (igual à SVC) e *acc/good* (3 instâncias). A classe mais difícil para o RF é *good*: é a menor do dataset ($\approx 4\%$ das instâncias) e exige a combinação mais específica de atributos no modelo DEX. Errar na fronteira *acc/good* mostra que o RF tem mais dificuldade com classes menos representadas.

Instâncias confusas. Os erros ocorrem sempre entre classes ordinalmente adjacentes. As instâncias na fronteira do modelo DEX são aquelas em que quase tudo aponta para uma classe melhor, mas um único fator (custo de compra ou manutenção muito alto, por exemplo) puxa para a classe inferior. Essas instâncias se parecem muito com as da classe vizinha, o que torna as confusões *unacc* $\rightarrow$ *acc* e *acc* $\rightarrow$ *good* muito mais prováveis do que um salto de duas ou mais posições.

6.5 Modelo selecionado

O MLP é o melhor modelo em todas as métricas e não erra nenhuma instância no conjunto de teste. Ela é a escolha natural para a etapa de interpretabilidade na próxima seção (Seção 7).

7 Interpretação do Melhor Modelo Treinado

O MLP, por ser uma rede neural, não oferece interpretação direta da sua lógica de decisão — é uma caixa-preta por natureza. Para entender quais atributos são mais relevantes e como eles influenciam as previsões, usamos dois métodos complementares, ambos agnósticos ao modelo: **Importância por Permutação** e **SHAP** (*SHapley Additive exPlanations*) [6]. A análise usa o MLP treinada no conjunto de treino completo — a mesma que atingiu 100% no teste.

7.1 Importância por Permutação

A ideia é direta: embaralhamos os valores de um atributo (quebrando a relação entre ele e o rótulo) e medimos o quanto o F1-macro do modelo cai. Atributo muito importante → queda grande; atributo irrelevante → pouca mudança. O processo é repetido 30 vezes para reduzir o efeito da aleatoriedade do embaralhamento.

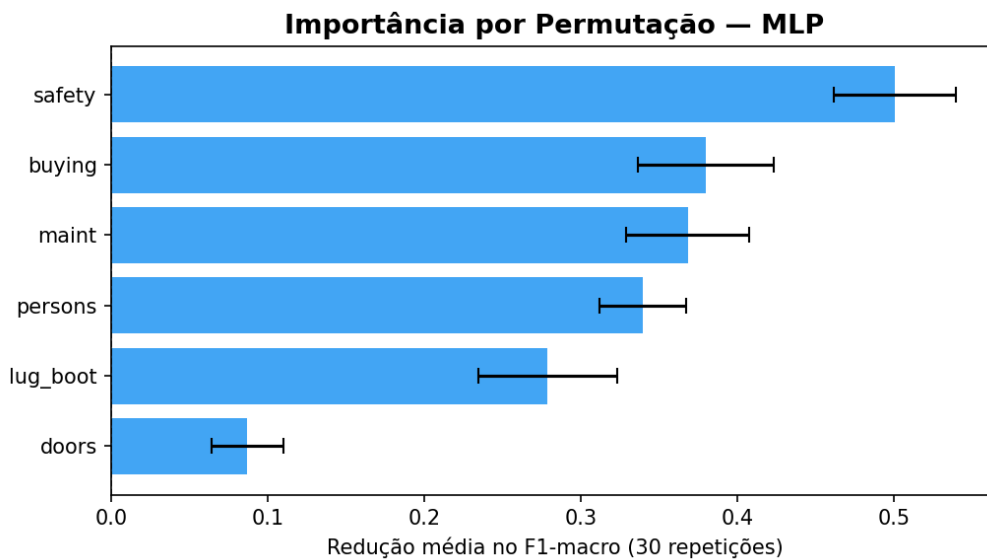


Figura 28: Importância por Permutação dos seis atributos para o MLP (F1-macro, 30 repetições). Barras de erro indicam o desvio padrão entre repetições.

O resultado mostra uma separação clara entre os atributos: **safety** e **persons** são os mais importantes por larga margem. Em seguida aparecem os fatores de custo, **buying** e **maint**, com importâncias similares entre si. **lug_boot** fica em penúltimo e **doors** tem a menor importância de todos.

Esse ranking faz sentido quando olhamos para as regras DEX: **safety=low** e **persons=2** já são suficientes para classificar como **unacc**, qualquer que seja o valor dos outros atributos. Embaralhar esses dois destrói a informação mais decisiva do modelo e, por isso, causa o maior impacto no desempenho.

7.2 Análise SHAP

SHAP atribui a cada par (instância, atributo) um valor que indica a contribuição daquele atributo para deslocar a previsão em relação à previsão média do modelo. É mais informativo que a

importância por permutação porque mostra não só *quanto* cada atributo importa, mas também *em que direção* ele empurra a previsão para cada instância individualmente.

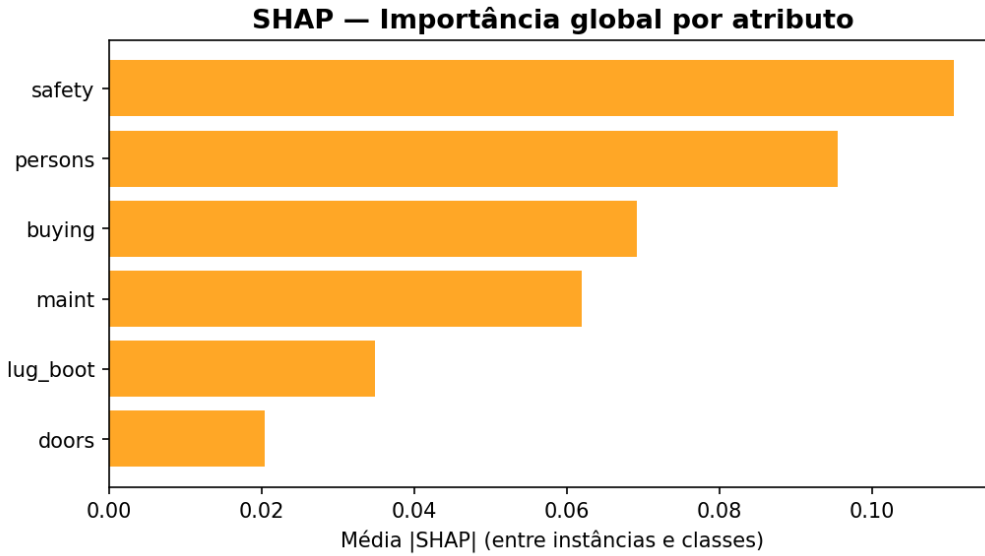


Figura 29: Importância global SHAP: média do valor absoluto das contribuições por atributo, agregada entre instâncias e classes.

A Figura 29 confirma o ranking da importância por permutação: **safety** e **persons** lideram, seguidos pelos atributos de custo e, por último, **lug_boot** e **doors**.

A Figura 30 mostra o *beeswarm* SHAP para a classe **unacc** — a mais frequente e com as regras DEX mais diretas. Cada ponto representa uma instância do conjunto de teste. A posição horizontal indica a magnitude e o sinal da contribuição SHAP (positivo = empurra para **unacc**; negativo = empurra para longe de **unacc**). A cor representa o valor codificado do atributo: vermelho para valores altos, azul para valores baixos.

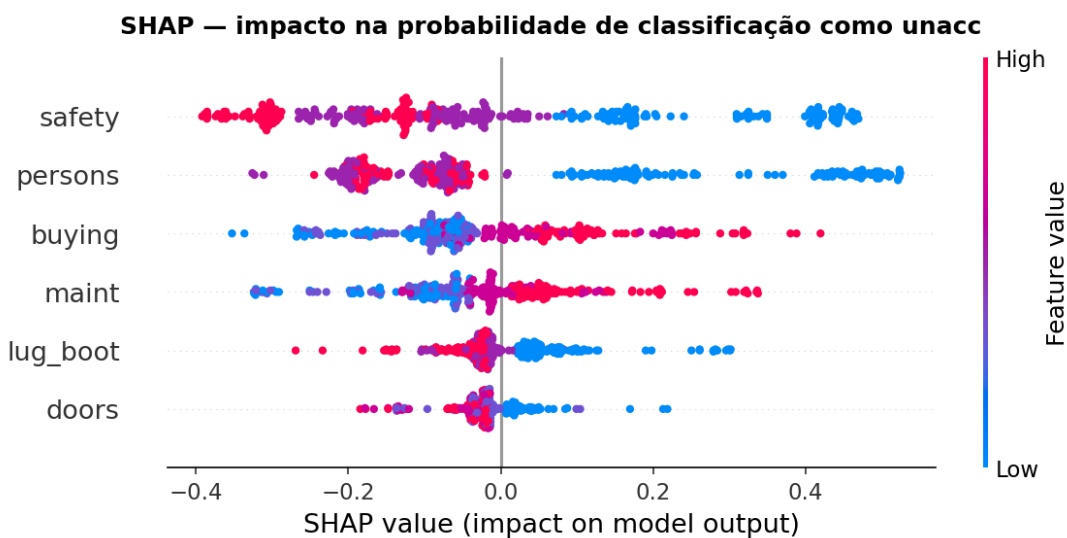


Figura 30: SHAP beeswarm para a classe **unacc**. Valores positivos empurram a previsão *em direção* a **unacc**; negativos, para longe dela. Cor: vermelho = valor alto do atributo, azul = valor baixo.

Alguns padrões se destacam no beeswarm:

- **safety**: instâncias com **safety=low** (valor baixo, azul) têm SHAP fortemente positivo, ou seja, o modelo as empurra com força para **unacc**. Já **safety=high** (vermelho) puxa na direção oposta, reduzindo bastante a probabilidade de inaceitável. Isso reflete diretamente a regra DEX: segurança baixa já basta para o carro ser inaceitável.
- **persons**: o padrão é análogo. **persons=2** (azul) → forte contribuição para **unacc**; mais vagas disponíveis → contribuição negativa. Poucas vagas é, junto com segurança, uma das restrições mais duras do modelo DEX.
- **buying** e **maint**: valores altos (vermelho, custo elevado) contribuem positivamente para **unacc**, mas com menor magnitude e mais dispersão — custo alto aumenta a probabilidade de inaceitável, mas não é uma condição absoluta como segurança e lotação.
- **lug_boot** e **doors**: contribuições próximas de zero na grande maioria das instâncias, confirmando que são os atributos menos determinantes para a classificação.

7.3 Discussão

Os dois métodos chegam à mesma conclusão: o MLP aprendeu a hierarquia das regras DEX, com **safety** e **persons** como filtros principais e os fatores de custo como critério secundário. O beeswarm vai além e mostra a direção dessas relações: mais segurança → menos **unacc**; custo mais alto → mais **unacc**. Faz sentido, já que os atributos são ordinais e foram codificados nessa ordem.

O resultado mais importante é que o ranking aprendido pelo MLP bate com a hierarquia das regras DEX — uma rede que nunca viu as regras diretamente, só os dados gerados por elas, acabou reproduzindo a mesma lógica de decisão. Isso mostra que o MLP realmente aprendeu a função do sistema especialista, não só memorizou os exemplos.

O resultado mais curioso é o papel de **doors**: mesmo sendo um dos seis atributos, o MLP aprendeu que ele quase não muda a previsão. Faz sentido no modelo DEX, onde o número de portas é secundário dentro do critério de conforto — mas pode surpreender quem espera que todo atributo do dataset pese na decisão.

8 Conclusão

O trabalho cobriu todo o pipeline pedido, com análise exploratória, pré-processamento de dados, spot-checking (de 8 algoritmos), otimização de hiperparâmetros com *nested cross-validation*, comparação de desempenho sobre os melhores modelos encontrados em dados nunca antes vistos e análise de interpretabilidade sobre o melhor modelo.

O *dataset* foi bastante simples de ser aprendido pelos modelos, devido ao fato de que não há ruídos e de que ele foi construído por regras DEX, mas permitiu vermos empiricamente que os métodos de análise de importância de atributos realmente são eficazes, comparando os resultados da análise com o que foi observado na análise exploratória.

8.1 Melhor modelo obtido

O MLP foi o melhor modelo em todas as métricas: F1-macro de 1,0 e zero erros nas 346 instâncias de teste. A SVC ficou em segundo com F1-macro de 0,9875 (6 erros), e o *Random Forest* em terceiro com 0,9566 (9 erros). Os três modelos apresentam desempenho muito superior ao observado no spot-checking, confirmando que a otimização de hiperparâmetros teve impacto significativo, especialmente para a SVC, que saltou de $\approx 0,89$ para $\approx 0,99$.

O MLP alcançou um resultado perfeito pois o *dataset* foi construído com a hierarquia de regras DEX, sem nenhum ruído. Pelo fato de não haver ruído nos dados (cada instância era uma instância correta do problema), o MLP aprendeu a função exata, não havendo overfitting. A análise de interpretabilidade confirma que o MLP internalizou a hierarquia do modelo DEX: **safety** e **persons** como fatores determinantes, custo como critério secundário, e **doors** com influência quase nula.

8.2 Desafios encontrados

O principal desafio foi o tempo de otimização, cujo processo levou cerca de quatro horas. O grande espaço de busca do MLP (mais de 20 hiperparâmetros) foi o maior responsável por isso.

A análise SHAP com KernelExplainer também foi custosa, pois o método é agnóstico ao modelo e precisa avaliar muitas combinações de atributos para estimar as contribuições. No caso do Car Evaluation, o custo foi aceitável pelo tamanho pequeno do *dataset*, mas em *datasets* maiores essa abordagem poderia ser inviável.

Por fim, o resultado de 100% de acerto do MLP nos dados de teste gerou dúvida inicial sobre overfitting, que precisou ser analisada com cuidado para ser descartada com base na natureza determinística do *dataset*.

8.3 Futuras possibilidades

Uma extensão natural do trabalho seria aplicar o mesmo pipeline a uma versão ruidosa do *dataset*, adicionando ruído artificial às respostas, para verificar se os modelos mantêm o desempenho quando a função-alvo não é mais perfeitamente aprendível. Isso testaria o quão robusto cada modelo realmente é.

Também acreditamos que a análise de interpretabilidade poderia ser estendida com métodos que analisem a importância de atributos dependentes uns dos outros, como LIME ou análise de interação entre atributos, para verificar se o MLP captou não só a importância individual de cada atributo, mas também combinações relevantes como **safety=low** e **persons=2** em conjunto.

Declaração de Uso de IA

Declaramos que o uso de agentes de inteligência artificial foi bastante útil no desenvolvimento deste trabalho, sem, entretanto, participar das tomadas de decisão dos autores. Os agentes auxiliaram em aspectos relacionados à sintaxe de bibliotecas como *Pandas*, *scikit-learn* e *Matplotlib*, na organização do relatório (incluindo auxílio com o ambiente L^AT_EX e com padrões de relatórios acadêmicos) e, principalmente, na compreensão de modelos que não constam no conteúdo da disciplina, mas que

optamos por analisar, como Naive Bayes e SVM. Também requisitamos aos agentes as referências bibliográficas utilizadas em suas respostas, com o objetivo de verificar a origem das informações fornecidas e, em alguns casos, consultar diretamente o material original.

Reiteramos que as tomadas de decisão relacionadas à análise exploratória dos dados, ao pré-processamento, à escolha dos algoritmos e métricas utilizadas no spot-checking, bem como às conclusões apresentadas, foram realizadas e discutidas entre os membros do grupo.

Os agentes utilizados foram o ChatGPT, usado principalmente para dúvidas mais simples e pontuais, como aquelas relacionadas à documentação de bibliotecas, e o Gemini, utilizado para auxiliar em questões mais complexas, especialmente envolvendo o funcionamento dos algoritmos Naive Bayes e SVM.

Referências

- [1] Bohanec, M. & Rajkovič, V. (1988). *Knowledge acquisition and explanation for multi-attribute decision making*. 8th Intl. Workshop on Expert Systems and their Applications, Avignon, France, pp. 59–78.
- [2] Dua, D. & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine. <https://archive.ics.uci.edu/dataset/19/car+evaluation>
- [3] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [4] Faceli, K., Lorena, A. C., Gama, J. & Carvalho, A. C. P. L. F. (2021). *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina* (2ª ed.). LTC.
- [5] Géron, A. (2022). *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. Sebastopol, California: O'Reilly Media, Inc. ISBN 9798228679245.
- [6] Lundberg, S. M. & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.